

SAT-Based Space Partitioning and Applications to Ascon-Hash256 Cryptanalysis

Guozhen Liu^{1,2}, Shun Li^{1,2}, Huina Li^{3,4(✉)}, Weidong Qiu⁴ and Siwei Sun^{1,2}

¹ School of Cryptology, University of Chinese Academy of Sciences, Beijing, 100049, China

{liuguozhen, lishun, sunsiwei}@ucas.ac.cn

² State Key Laboratory of Cryptology, P.O. Box 5159, Beijing, 100878, China

³ Tsinghua University, Beijing, 100084, China

lihuina@mail.tsinghua.edu.cn

⁴ Shanghai Jiao Tong University, Shanghai, 200240, China

qiuwd@sjtu.edu.cn

Abstract. We introduce an efficient SAT-based space partitioning technique that enables systematic exploration of large search spaces in cryptanalysis. The approach divides complex search spaces into manageable subsets through combinatorial necklace generation, allowing precise tracking of explored regions while maintaining search completeness.

We demonstrate the technique’s effectiveness through extensive cryptanalysis of **Ascon-Hash256**. For differential-based collision attacks, we conduct an exhaustive search of 2-round collision trails, proving that no collision trail with weight less than 156 exists. Through detailed complexity analysis and parameter optimization, we present an improved 2-round collision attack with complexity $2^{61.79}$. We also discover new Semi-Free-Start (SFS) collision trails that enable practical attacks on both 3-round and 4-round **Ascon-Hash256**, especially improving the best known 4-round SFS trail from weight 295 to 250.

Furthermore, applying the technique to Meet-in-the-Middle structure search yields improved attacks on 3-round **Ascon-Hash256**. We reduce the collision attack complexity from $2^{116.74}$ to $2^{114.13}$ with memory complexity 2^{112} (improved from 2^{116}), and the preimage attack complexity from $2^{162.80}$ to $2^{160.75}$ with memory complexity 2^{160} (improved from 2^{162}).

Keywords: SAT · Space Partitioning · **Ascon-Hash256** · Collision Attack · Preimage Attack · Meet-in-the-Middle

1 Introduction

In 2018, the National Institute of Standards and Technology (NIST) initiated a competition [NIS18] to standardize lightweight cryptographic algorithms, including authenticated encryption with associated data (AEAD) and hash functions designed for resource-constrained environments. Following rigorous evaluation, the Ascon [DEMS21] family was selected as the winner in February 2023. In August 2025, NIST published the final standard [STMC⁺25] specifying the AEAD scheme **Ascon-AEAD128**, the hash function **Ascon-Hash256**, and the eXtensible Output Functions **Ascon-XOF128** and **Ascon-CXOF128**, all based on the Ascon permutation.

Cryptanalysis of the Ascon Permutation. The Ascon permutation has undergone extensive cryptanalysis, particularly focusing on establishing differential/linear bounds to evaluate its security. The primary approach involves finding high-probability differential

trails (also called differential characteristics) through detailed analysis of cryptographic properties.

Two main approaches have emerged for finding high-probability differential trails in the Ascon permutation. The first utilizes automatic search tools based on mathematical optimization problems, including Satisfiability Modulo Theories (SMT) [DEMS15, BM22], Mixed Integer Linear Programming (MILP) [DDLS25], Constraint Programming (CP) [GPT21], and Boolean Satisfiability Problem (SAT) [EME22]. The second approach employs dedicated search tools [DEMS19, HMMD22, MR22] following Silvia *et al.*'s [MDA17, MDA23] tree-based search algorithm.

Overall, the Ascon permutation has shown strong resistance against known cryptanalytic techniques.

Cryptanalysis of Ascon-Hash256. Cryptanalytic results against Ascon-Hash256 remain limited. In [DEMS19], Dobraunig *et al.* gave the first practical 4-round Semi-Free-Start (SFS) collision attack¹ using a 4-round differential trail with weight 295². Subsequently, Gerault *et al.* [GPT21] developed a CP model to identify a 2-round differential trail with weight 156, leading to a collision attack with complexity 2^{103} . Yu *et al.* [YLW⁺23] later improved upon this attack, reducing the time complexity to $2^{62.6}$ through an optimized attack framework.

More recently, Dong *et al.* [DZQ⁺24] introduced a generalized Meet-in-the-Middle (MitM) attack framework for Sponge constructions [BDPVA07]. They presented the first collision attacks on 3-/4-round Ascon-Hash256 with time complexities of $2^{116.74}$ and $2^{124.85}$ respectively, and corresponding memory complexities of 2^{116} and 2^{124} . Additionally, they proposed the first MitM preimage attacks on 3-/4-/5-round Ascon-Hash256, achieving time complexities of $2^{162.80}$, $2^{184.85}$, and $2^{191.31}$, with memory complexities of 2^{160} , 2^{178} , and 2^{190} respectively.

Motivation. In recent years, automatic search tools based on MILP, CP, SMT, and SAT have become increasingly prevalent in cryptanalysis for generating structures used in various attack scenarios. SAT-based methods, in particular, have emerged as powerful tools [SWW21, GLST22, LLW24], showing effectiveness for bit-oriented primitives [BPF⁺23, DDLS25] and primitives with large state sizes, as demonstrated in collision attacks on SHA-3 [GLST22].

However, all automated tools, including SAT solvers, face a significant challenge when the search space is vast and solutions are sparse. In such scenarios, solvers often become stuck in an indefinite solving process without producing results in a feasible amount of time, regardless of whether they are running on a single thread or in parallel. This stagnation has become a common issue in cryptanalysis, representing one of the main obstacles to the effectiveness of automated search tools.

One approach to address this limitation is search space partitioning. In ToSC 2022, Erlacher *et al.* [EME22] introduced a pioneering partitioning strategy. Their approach divides the search space into smaller subspaces by fixing active Sbox patterns, which is analogous to using two-colored (active/inactive) necklaces. This allows solvers to tackle each subspace individually.

While innovative, their technique's scalability is limited when applied to extremely large spaces, as fixing patterns can lead to an intractably large number of partitions. Inspired by their work, we propose a more advanced and efficient partitioning technique. By employing multi-colored necklaces, our method offers a more granular and scalable way

¹In an SFS collision attack, the attacker can choose both the initial chain value (IV) and a pair of distinct messages M_1, M_2 such that $H(IV, M_1) = H(IV, M_2)$.

²Throughout this paper, *propagation weight* (or simply *weight*) refers to the negative base-2 logarithm of the estimated differential probability.

to decompose the search space, enabling the exploration of problems that were previously out of reach.

Table 1: Summary of Preimage and (SFS/FS) Collision Attacks on Ascon-Hash256 and Ascon-XOF.

Target	Hash	Type	Round	Time	Memory	Method	Reference
Ascon-Hash256	256	Collision	2	$2^{61.79}$	$\neg \dagger$	Diff.	Sect. 4.1*
			2	$2^{62.6}$	$\neg \dagger$	Diff.	[YLW ⁺ 23]
			3	$2^{114.13}$	2^{112}	MitM	Sect. 5.1*
			3	$2^{116.74}$	2^{116}	MitM	[DZQ ⁺ 24]
			4	$2^{124.85}$	2^{124}	MitM	[DZQ ⁺ 24]
		SFS Collision	3	Practical	$\neg \dagger$	Diff. (216)	Sect. 4.3*
			4	Practical	$\neg \dagger$	Diff. (250)	Sect. 4.3*
			4	Practical	$\neg \dagger$	Diff. (295)	[DEMS19]
		FS Collision	5	2^{41}	$\neg \dagger$	Diff.	[FLYS25]
		Preimage	3	$2^{160.75}$	2^{160}	MitM	Sect. 5.2*
			3	$2^{162.80}$	2^{160}	MitM	[DZQ ⁺ 24]
			4	$2^{184.85}$	2^{178}	MitM	[DZQ ⁺ 24]
			5	$2^{191.31}$	2^{190}	MitM	[DZQ ⁺ 24]
			5	$2^{191.31}$	2^{190}	MitM	[DZQ ⁺ 24]
Ascon-XOF	64	Preimage	2	$2^{31.56}$	$\neg \dagger$	Algebraic	[LHC ⁺ 23]
			3	2^{56}	$\neg \dagger$	Algebraic	[BKK24]
			4	2^{63}	$\neg \dagger$	Algebraic	[BKK24]
	128	Preimage	2	2^{97}	$\neg \dagger$	Algebraic	[FLYS25]
			3	$2^{112.21}$	$\neg \dagger$	Algebraic	[LHC ⁺ 23]
			4	$2^{124.49}$	$\neg \dagger$	Algebraic	[LHC ⁺ 23]

* This work.

$\dagger$ Negligible memory requirements.

Our Contribution. This paper presents a novel SAT-based search space partitioning technique that enables efficient exploration of large search spaces. We apply this technique to both differential collision trail search and MitM structure search for Ascon-Hash256, achieving improved collision and preimage attacks. For the source codes, please refer to <https://github.com/HuinaLi/SAT-Based-Space-Partitioning>. Our main contributions are:

- **A New Search Space Partitioning Technique.** We introduce an efficient SAT-based search technique that partitions large search spaces into manageable subspaces. By modeling search problems as combinatorial necklace generation problems, our approach enables effective space partitioning while maintaining search completeness. Additionally, we analyze how different SAT encoding strategies affect solver efficiency, which is an aspect often overlooked in SAT-based cryptanalytic applications. We investigate two SAT encoding strategies in our implementation, i.e., comprehensive variable relationship encoding and elimination of redundant variables. Our experimental results show that these strategies significantly improve the solving efficiency in our case, suggesting potential benefits for other SAT-based cryptanalysis applications.
- **Improved Cryptanalysis against Ascon-Hash256.** We apply the space partitioning technique to both the differential-based collision trail search and the MitM structure search against Ascon-Hash256.

For the collision trail search, we conduct an exhaustive exploration of 2-round collision trails, proving that no trail with weight less than 156 exists. Through parameter

optimization, we achieve an improved collision attack with complexity $2^{61.79}$. We also extend our search to 3-/4-/5-round collision trails, obtaining practical SFS collision attacks on 3-/4-round **Ascon-Hash256**.

For the MitM structure search, we discover improved MitM collision and preimage structures, leading to better attacks against 3-round **Ascon-Hash256**. Specifically, we improve the collision attack time complexity from $2^{116.74}$ to $2^{114.13}$ and memory complexity from 2^{116} to 2^{112} , and the preimage attack time complexity from $2^{162.80}$ to $2^{160.75}$ and memory complexity from 2^{162} to 2^{160} .

Table 1 provides a comprehensive comparison of current state-of-the-art preimage, collision, Semi-Free-Start(SFS) collision, and Free-Start(FS) collision attacks on **Ascon-Hash256** and **Ascon-XOF** (solely for completeness).

Outline. Section 2 provides an overview of Ascon, introduces attack frameworks for cryptanalysis, and presents a brief introduction to the SAT-based automatic search. Section 3 introduces a space partitioning technique inspired by necklace generation in combinatorics to efficiently explore the search space. Section 4 focuses on differential-based collision attacks against **Ascon-Hash256**, with an emphasis on 2-round attacks and extending to 3-/4-/5-round collision trails. Section 5 applies the partitioning technique to find improved MitM structures, leading to better collision and preimage attacks on 3-round **Ascon-Hash256**. Finally, Section 6 summarizes the findings.

2 Preliminary

2.1 Description of Ascon

Ascon consists of AEAD and hashing schemes, both built upon the Ascon permutation, e.g., p^a and p^b with rounds $a = 12$ and $b = 8$. In this paper, we focus on the hash function **Ascon-Hash256**. The round function f consists of three steps p_C , p_S , and p_L , denoted as $f = p_L \circ p_S \circ p_C$. The Ascon permutation operates on a 320-bit state S arranged into five rows, i.e., $S = w_0 \| w_1 \| w_2 \| w_3 \| w_4$. Each row is a 64-bit word stored in the big-endian mode.

Addition of Constants (p_C). p_C adds a round constant c_i to register word w_2 of the state S in round i . The round constants c_i are shown in Table 2.

Table 2: Constants c_i used in Ascon permutation.

Round i	Constant c_i	Round i	Constant c_i
0	000000000000000000f0	6	000000000000000000096
1	0000000000000000000e1	7	000000000000000000087
2	0000000000000000000d2	8	000000000000000000078
3	0000000000000000000c3	9	000000000000000000069
4	0000000000000000000b4	10	00000000000000000005a
5	0000000000000000000a5	11	00000000000000000004b

Substitution Layer (p_S). p_S operates on the state S with 64 parallel applications of the 5-bit Sbox to each bit-slice of the five registers w_0, w_1, w_2, w_3, w_4 . We denote the input and output of the Sbox by $(x_0, x_1, x_2, x_3, x_4)$ and $(y_0, y_1, y_2, y_3, y_4)$, respectively. The ANF

of the 5-bit Sbox is given by

$$\begin{aligned}
y_0 &= x_4x_1 \oplus x_3 \oplus x_2x_1 \oplus x_2 \oplus x_1x_0 \oplus x_1 \oplus x_0 \\
y_1 &= x_4 \oplus x_3x_2 \oplus x_3x_1 \oplus x_3 \oplus x_2x_1 \oplus x_2 \oplus x_1 \oplus x_0 \\
y_2 &= x_4x_3 \oplus x_4 \oplus x_2 \oplus x_1 \oplus 1 \\
y_3 &= x_4x_0 \oplus x_4 \oplus x_3x_0 \oplus x_3 \oplus x_2 \oplus x_1 \oplus x_0 \\
y_4 &= x_4x_1 \oplus x_4 \oplus x_3 \oplus x_1x_0 \oplus x_1.
\end{aligned} \tag{1}$$

Linear Diffusion Layer (p_L). p_L provides diffusion within each 64-bit register word w_i , $0 \leq i < 5$, as follows

$$\begin{aligned}
w_0 &\leftarrow \Sigma_0(w_0) = w_0 \oplus (w_0 \ggg 19) \oplus (w_0 \ggg 28) \\
w_1 &\leftarrow \Sigma_1(w_1) = w_1 \oplus (w_1 \ggg 61) \oplus (w_1 \ggg 39) \\
w_2 &\leftarrow \Sigma_2(w_2) = w_2 \oplus (w_2 \ggg 1) \oplus (w_2 \ggg 6) \\
w_3 &\leftarrow \Sigma_3(w_3) = w_3 \oplus (w_3 \ggg 10) \oplus (w_3 \ggg 17) \\
w_4 &\leftarrow \Sigma_4(w_4) = w_4 \oplus (w_4 \ggg 7) \oplus (w_4 \ggg 41).
\end{aligned}$$

Ascon-Hash256. In this paper, we explore attacks on Ascon-Hash256. The sponge construction, as illustrated in Figure 1, operates on a 320-bit internal state which is split into two parts, i.e., the r -bit outer part ($r = 64$) and the c -bit inner part ($c = 256$).

During the initialization phase, the 320-bit initial state of Ascon-Hash256 is specified as a 64-bit constant $IV = 0x0000080100cc0002$ and a 256-bit zero value. The 12-round permutation p^{12} is applied to initialize a 320-bit initial state for the next absorbing phase.

Then, the sponge construction processes the message M in two phases. In the *absorbing phase*, the message M is first padded with a 10^* string such that M is split into n blocks of r bits, i.e., $M_0 || \dots || M_{n-1}$. These message blocks are processed iteratively by XORing each block into the first r bits of the current state, and then applying the p^{12} permutation to the 320-bit state.

After processing all the blocks, the sponge construction switches to the *squeezing phase*, where d -bit outputs are produced iteratively. In each iteration, the first 64 bits are extracted from the state and the permutation p^{12} is applied.

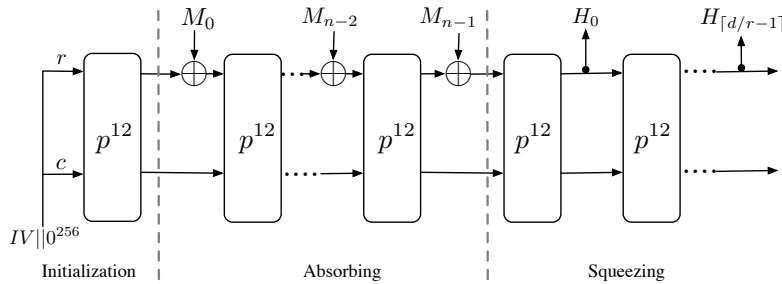


Figure 1: Illustration of Ascon-Hash256.

2.2 Overview of Attack Frameworks

In this work, we explore two types of attacks on Ascon-Hash256: differential-based collision attacks and MitM-based attacks. Both require finding specific cryptographic structures that are too complex to discover manually, necessitating automated search techniques.

2.2.1 Differential-based Collision Attack

The differential-based collision attack employs differential trails with relatively low probability to find message pairs that produce identical hash outputs. Since a single message block provides insufficient Degrees of Freedom (DoFs) to fulfill such trails, we use a multi-block framework where initial blocks provide additional DoFs.

As shown in Figure 2, given $s + 1$ message blocks M_i ($i \in \{0, \dots, s\}$), the first $s - 1$ blocks have no differences and serve to provide DoFs. At block M_{s-1} , we introduce a difference ΔM_{s-1} . An r -round differential trail is constructed for the permutation rounds that process this block. The trail must ensure zero difference in the inner part of its output. Finally, at block M_s , we cancel the remaining differences to achieve a collision.

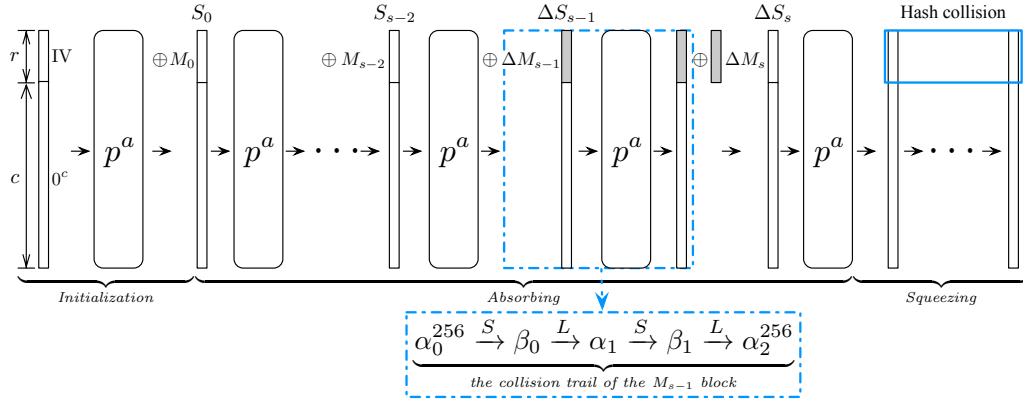


Figure 2: The multi-block collision attack framework of Ascon-Hash256.

Differential Trail in Collision Attack. The construction of effective differential trails for sponge-based hash functions faces the Constrained-Input Constrained-Output (CICO) problem [BDPVA07]. For the differential trail applied during the processing of block M_{s-1} , CICO introduces constraints on both the input and output differences. Specifically, the inner part of the input difference ΔS_{s-1} and the output difference are constrained to zero.

A differential trail for a collision attack is typically conceptualized as having two components, i.e., a *connecting trail* and a *colliding trail*. The connecting trail constitutes the initial part of the trail that satisfies the input difference constraints, while the colliding trail is the subsequent part that fulfills the output difference constraints. The concatenation of these two components forms the complete *collision trail*.

Take a 5-round collision trail as an example. With k indexed from 0, let α_k denote the input difference to the k -th round Sbox, and β_k denote the input difference to the k -th round p_L operation. The inner parts of both the input and output differences are zero, denoted as α_0^c and α_5^c , respectively.

$$\begin{array}{c}
 \text{the } n_{r_2}\text{-round connecting trail} \\
 \alpha_0^c \xrightarrow{S} \beta_0 \xrightarrow{L} \alpha_1 \xrightarrow{S} \beta_1 \xrightarrow{L} \alpha_2 \xrightarrow{S} \beta_2 \xrightarrow{L} \alpha_3 \xrightarrow{S} \beta_3 \xrightarrow{L} \alpha_4 \xrightarrow{S} \beta_4 \xrightarrow{L} \alpha_5^c \\
 \text{the } n_{r_1}\text{-round colliding trail}
 \end{array}$$

2.2.2 MitM-based Collision and Preimage Attacks

Both collision and preimage attacks against Ascon-Hash256 can be constructed using MitM structures, a powerful cryptanalytic technique with a rich history of applications [DHS⁺21] and automated search methods [BDG⁺21]. In this work, we adopt the established attack frameworks from [DZQ⁺24] for both attack types. Since the framework details are

extensively covered in their work, we focus here on finding improved MitM structures rather than modifying the attack framework itself.

The MitM Structures. To understand how MitM attacks work, we first describe the fundamental MitM structure that underlies both collision and preimage attacks. The MitM structure divides the initial state bits into different parts, typically denoted as red bits, blue bits, and gray bits (representing constants). The red and blue parts then propagate in different ways. This includes forward and backward propagation in classical MitM attacks, or forward propagation in the same direction for MitM frameworks applied to sponge-based constructions.

After propagation, the red and blue bits meet at a matching point governed by specific matching conditions. At this matching point, red and blue matching values are independently computed and compared according to the matching conditions. When a match is found, the corresponding initial red value and blue value can be concatenated to form a target message.

In this way, the MitM structure significantly reduces the time complexity required to obtain target messages, which would otherwise require exhaustive message search. The saved computational resources can then be allocated to other attack components, thereby enabling valid attacks.

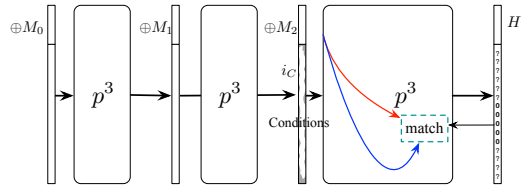


Figure 3: The MitM collision attack framework of 3-round Ascon-Hash256.

MitM Collision Attacks on Ascon-Hash256. The MitM collision attack adopts a framework similar to that depicted in Figure 2, where the MitM structure is applied to the processing of the final block M_2 . As shown in Figure 3, this structure identifies messages (M_0, M_1, M_2) that contain d_M fixed zeros in the inner part of the output. Preparing $2^{\frac{h-d_M}{2}}$ such messages statistically ensures a collision, where h represents the hash length.

MitM Preimage Attacks on Ascon-Hash256. The MitM preimage attack follows a two-phase framework illustrated in Figure 4. Phase I employs the MitM procedure to find partial preimages for a subset of the hash output, while Phase II uses generalized collision techniques or MitM procedures to find the inner collision (with a time complexity $\leq 2^{\frac{c}{2}}$), and hence complete the full preimage attack.

Interested readers can refer to [DZQ⁺24] for the complete framework descriptions.

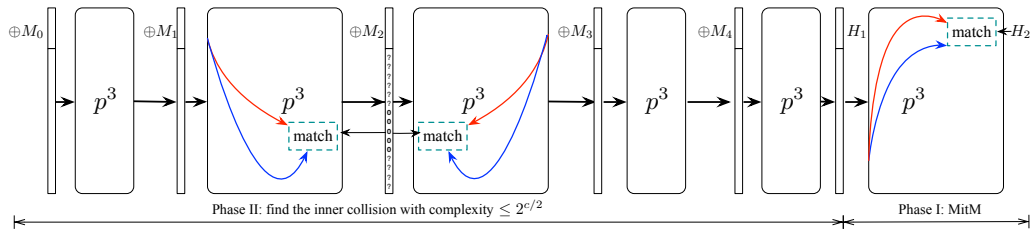


Figure 4: The MitM preimage attack framework of 3-round Ascon-Hash256.

2.3 SAT-Based Automatic Search

Modern cryptanalysis increasingly relies on automated tools to discover attack-enabling structures. For **Ascon-Hash256**, our focus is on finding both collision trails for differential-based attacks and MitM structures with optimal matching conditions.

Overview of SAT-Based Cryptanalysis. Among available tools, SAT-based approaches have emerged as particularly powerful due to their ability to efficiently handle complex boolean constraints. The SAT problem determines whether a given boolean formula has a satisfying assignment.

In cryptanalysis, we encode our search targets (such as differential trails or MitM structures) as boolean formulas, allowing SAT solvers to search for solutions. The basic SAT encoding methods are given in Supplementary Material A for interested readers.

SAT-based approaches have demonstrated particular effectiveness for bit-oriented primitives, often outperforming other automated tools such as MILP or CP when dealing with large state sizes.

Challenges in Large Search Spaces. While SAT tools excel at exploring complex search spaces, they face significant challenges when the space becomes extremely large. Two primary issues arise:

- **Resource Exhaustion:** For very large spaces, solvers may require impractical amounts of memory or computation time. For example, searching for 3-round differential and linear bounds of Xoodoo with SAT required about two months using 16 parallel threads [LZ22], compared to 16 core-days for dedicated tools [DHAK18].
- **Solver Stagnation:** When the search space is vast with few or no solutions, solvers may become stuck in an indefinite solving process without returning results, providing no indication of progress or completion.

Existing Space Management Approaches. Previous work has attempted to address these challenges through search space partitioning. For instance, Erlacher *et al.* [EME22] divided the search space by fixing active Sbox patterns in specific rounds. However, this approach becomes impractical when the number of patterns grows large (e.g., exceeding 2^{50} for more than 22 active Sboxes).

The limitations of existing approaches highlight the need for more sophisticated space management techniques that can systematically partition large search spaces while maintaining search completeness and providing clear progress indicators throughout the process, all while minimizing computational requirements. In the following sections, we present a novel space partitioning technique that addresses these challenges through combinatorial methods and enhanced SAT encoding strategies.

3 Search Space Partitioning with Necklace Generation

Different search tasks operate on distinct search spaces, e.g., differential trail search explores active patterns, MitM structure search examines state bit assignments. Yet they share a common challenge, i.e., systematically partitioning large spaces into manageable subsets.

We develop a space partitioning technique based on combinatorial necklace generation to enable systematic exploration while maintaining completeness guarantees. To demonstrate the approach, we focus on partitioning the space of 2-round collision trail search (which reduces to $\beta_0 \xrightarrow{L} \alpha_1$), using the β_0 differences as a primary example.

3.1 Partitioning the Search Space of β_0 Using Active Patterns

A common search strategy is to classify differences by the number of active Sboxes, following which we examine the distribution of active Sboxes through the concept of active Sbox patterns.

Definition 1 (*active Sbox pattern*). The *active Sbox pattern* (or *active pattern* for brevity) describes the distribution of active Sboxes among the 64 Sboxes in β_0 difference.

In a subspace of β_0 containing differences with exactly n active Sboxes, there are $\binom{64}{n}$ active patterns. Each pattern represents a set of 2^{2n} differences, as for each active Sbox, the corresponding input difference “10000” maps to 4 possible output differences that form the value in β_0 . Consequently, the total number of β_0 differences in the subspace with n active Sboxes is $2^{2n} \cdot \binom{64}{n}$.

Furthermore, due to the translation invariance of the Ascon permutation, rotating an active pattern left or right results in 64 equivalent patterns. Hence, the total number of distinct active patterns is reduced approximately by a factor of 64.

Table 3 lists the number of active Sboxes alongside the total and distinct counts of active patterns. Since $\binom{64}{n} = \binom{64}{64-n}$, only subspaces with $n \leq 32$ are shown.

Table 3: Number of active Sboxes and corresponding counts of active patterns (*#patterns*).

n	<i>#patterns</i>	n	<i>#patterns</i>	n	<i>#patterns</i>	n	<i>#patterns</i>
1	$2^{6.00} \rightarrow \mathbf{2^{0.00}}$	9	$2^{34.68} \rightarrow \mathbf{2^{28.68}}$	17	$2^{50.29} \rightarrow \mathbf{2^{44.29}}$	25	$2^{58.48} \rightarrow \mathbf{2^{52.48}}$
2	$2^{10.98} \rightarrow \mathbf{2^{4.98}}$	10	$2^{37.14} \rightarrow \mathbf{2^{31.14}}$	18	$2^{51.68} \rightarrow \mathbf{2^{45.68}}$	26	$2^{59.06} \rightarrow \mathbf{2^{53.06}}$
3	$2^{15.35} \rightarrow \mathbf{2^{9.35}}$	11	$2^{39.44} \rightarrow \mathbf{2^{33.44}}$	19	$2^{52.95} \rightarrow \mathbf{2^{46.95}}$	27	$2^{59.55} \rightarrow \mathbf{2^{53.55}}$
4	$2^{19.28} \rightarrow \mathbf{2^{13.28}}$	12	$2^{41.58} \rightarrow \mathbf{2^{35.58}}$	20	$2^{54.12} \rightarrow \mathbf{2^{48.12}}$	28	$2^{59.96} \rightarrow \mathbf{2^{53.96}}$
5	$2^{22.86} \rightarrow \mathbf{2^{16.86}}$	13	$2^{43.58} \rightarrow \mathbf{2^{37.58}}$	21	$2^{55.19} \rightarrow \mathbf{2^{49.19}}$	29	$2^{60.27} \rightarrow \mathbf{2^{54.27}}$
6	$2^{26.16} \rightarrow \mathbf{2^{20.16}}$	14	$2^{45.44} \rightarrow \mathbf{2^{39.44}}$	22	$2^{56.16} \rightarrow \mathbf{2^{50.16}}$	30	$2^{60.49} \rightarrow \mathbf{2^{54.49}}$
7	$2^{29.21} \rightarrow \mathbf{2^{23.21}}$	15	$2^{47.18} \rightarrow \mathbf{2^{41.18}}$	23	$2^{57.03} \rightarrow \mathbf{2^{51.03}}$	31	$2^{60.62} \rightarrow \mathbf{2^{54.62}}$
8	$2^{32.04} \rightarrow \mathbf{2^{26.04}}$	16	$2^{48.80} \rightarrow \mathbf{2^{42.80}}$	24	$2^{57.80} \rightarrow \mathbf{2^{51.80}}$	32	$2^{60.67} \rightarrow \mathbf{2^{54.67}}$

While this classification provides insight into the search space structure, directly enumerating all patterns becomes impractical for $n \geq 26$. Prior work [EME22] using necklace enumeration could only handle cases where $n \leq 8$. To address this limitation, we introduce a more sophisticated partitioning strategy in the next section.

3.2 Partitioning Active Patterns Using Necklace-based Combinations

We first introduce a grouping-based model that organizes adjacent active Sboxes into structured units.

Definition 2 (*active group*). An active group refers to a contiguous (i.e., adjacent) sequence of i active Sboxes within a pattern of n active Sboxes, where the first Sbox in the sequence must be preceded by an inactive Sbox.

The size of an active group, denoted i , satisfies $i \in \{1, 2, \dots, K\}$, where K is the maximum allowable size of an active group. Each active group is separated from neighboring groups by at least one inactive Sbox.

³The blue frame at the left margin appears larger than the right side to properly cover an active bead that spans across the pattern boundary, emphasizing that this is one active bead involving 3 Sboxes.

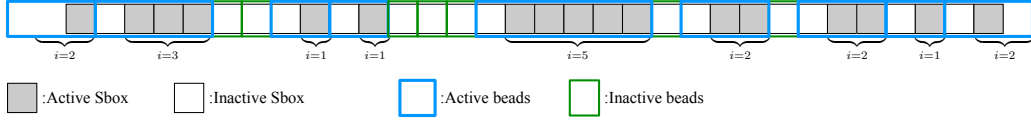


Figure 5: Illustration of active groups in a 32-bit active pattern³

Figure 5 illustrates this concept using a 32-bit pattern with $n = 17$ active Sboxes. The active Sboxes form groups of varying sizes: three groups of size 1, three groups of size 2, one group of size 3, and one group of size 5, giving a maximum group size $K = 5$.

To formalize this representation, let n_i denote the number of active groups containing exactly i active Sboxes. For the pattern shown in Figure 5, we have $n_1 = 3$, $n_2 = 3$, $n_3 = 1$, $n_4 = 0$, and $n_5 = 1$.

To systematically partition the search space using active groups, we introduce a formal way to describe patterns based on their group structure.

Definition 3 (combination of active groups). Given n (the number of active Sboxes) and K (the maximum group size), the *combination of active groups* (or simply *combination*) is defined as a vector $[n_1, n_2, \dots, n_K]$, where n_i specifies the number of active groups of size i ($i = 1, \dots, K$).

The combination vector must satisfy the following constraints: $n_K \geq 1$, ensuring at least one group of the maximum size is present, and $n_i \geq 0$ for $i \in \{1, \dots, K - 1\}$, indicating that groups of sizes smaller than K are not mandatory.

Each combination represents a distinct subset of active patterns that share the same group structure. By enumerating all valid combinations for given n and K , we can systematically partition the search space into non-overlapping subsets.

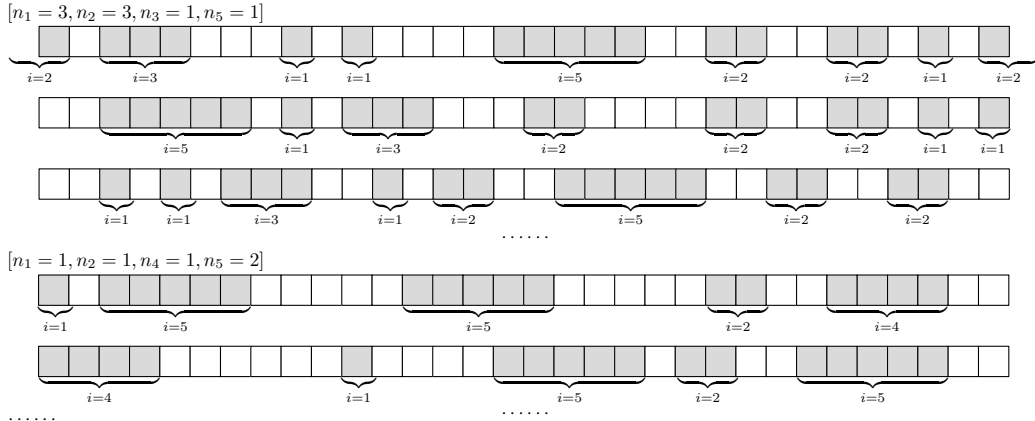


Figure 6: Combinations for 32-bit active patterns with $n = 17$ and $K = 5$.

Figure 6 illustrates several combinations of active groups. The combination vector $[n_1, n_2, \dots, n_K]$ captures the structural distribution of active Sboxes across groups of different sizes. For reference, combinations $[n_1 = 3, n_2 = 3, n_3 = 1, n_5 = 1]$ and $[n_1 = 1, n_2 = 1, n_4 = 1, n_5 = 2]$ are provided in this example.

For a given combination, there are multiple distinct active patterns that conform to the same group size configuration. For instance, with the combination $[n_1 = 3, n_2 = 3, n_3 = 1, n_5 = 1]$, the active groups can be arranged in various ways, demonstrating how this representation naturally partitions the subspace of active patterns based on their group structures.

Having established the theoretical framework of combinations, we now describe how to use them for systematic space partitioning.

Partitioning Subspace with Combinations of Active Groups. The analysis above demonstrates that the subspace of active patterns with n active Sboxes can be partitioned based on combinations of active groups. For given n and K , these combinations can be generated by enumerating all vectors $[n_1, n_2, \dots, n_K]$ that satisfy the following constraints.

- The relation between the total number of active Sboxes n and the combination vector, i.e., $\sum_{i=1}^K i \cdot n_i = n$. This ensures that the number of active Sboxes of the patterns within the combination is fixed to n .
- The conditions on the number of active groups, i.e., $n_K \geq 1$, and $n_i \geq 0$ for $i \in \{1, \dots, K-1\}$.

Given n , each active pattern within the subspace belongs to a unique combination. Different combinations correspond to distinct smaller subspaces of active patterns. Specifically, each combination $[n_1, n_2, \dots, n_K]$ defines a subset of the overall space of active patterns.

Thus, the subspace of active patterns with n active Sboxes is partitioned into disjoint subspaces, each represented by a unique combination. In summary, partitioning the space of β_0 using the combinations derived from n and K provides a comprehensive and systematic division.

Counting Distinct Patterns Within a Combination. The next task is to determine the number of active patterns within a subspace defined by a specific combination of active groups. This can be achieved using the combinatorial method of *necklace enumeration with fixed contents*, as proposed in [Saw03].

In combinatorial mathematics, a *necklace* refers to a circular arrangement of *beads* (or symbols), where arrangements that can be rotated into each other are considered equivalent. This property aligns with the translational invariance of active patterns.

In this context, each active group can be visualized as a bead, while the inactive Sboxes separating these groups can also be treated as beads. Consequently, an entire active pattern can be conceptualized as a necklace composed of active and inactive beads. The combination specifies the composition of the necklaces, including the types and sizes of beads.

To align active patterns with necklace representation, the description of each active group is extended to include its preceding inactive Sbox. Thus, each bead representing an active group of size i corresponds to $i + 1$ Sboxes, with the additional Sbox accounting for the preceding inactive Sbox. Inactive beads are those that do not directly precede any active group.

Based on this adjustment, we formally define *active beads* and *inactive beads* to facilitate the enumeration process.

Definition 4 (*active bead*). A bead representing any active group, including the preceding inactive Sbox, is called an *active bead*. Each active bead thus corresponds to $i + 1$ Sboxes as shown in Figure 5, highlighted by blue frames.

Definition 5 (*inactive bead*). Inactive Sboxes that are not associated with any active group are referred to as *inactive beads*. In Figure 5, inactive beads are depicted within green frames.

Denote b_i as the bead representing a group of i active Sboxes where $i \geq 0$. For instance, b_0 represents an inactive bead corresponding to a single inactive Sbox, while b_1 represents an active bead corresponding to a group of one active Sbox, and so forth. Also let n_{b_i} denote the number of b_i beads⁴.

⁴Note that n_{b_i} equals n_i for $i > 0$. We keep both notations to maintain clarity in different contexts: n_i when discussing active groups and n_{b_i} when describing necklace configurations.

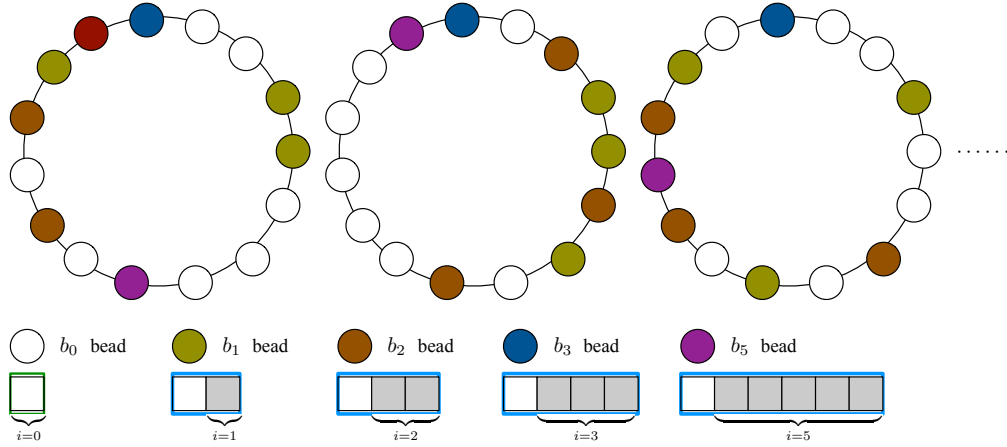


Figure 7: Necklaces of 15 beads for the combination $[n_1 = 3, n_2 = 3, n_3 = 1, n_5 = 1]$.

Figure 7 illustrates the composition of necklaces (or active patterns) for a specific combination. Each necklace consists of 15 beads, categorized into five types: seven inactive beads (b_0) and eight active beads, comprising three b_1 beads, three b_2 beads, one b_3 bead, and one b_5 bead. Three distinct necklaces are given to explain how unique active patterns are enumerated within the combination.

In general, active patterns within a combination $[n_1, n_2, \dots, n_K]$ of n active Sboxes can be represented as necklaces with $t = (64 - n)$ beads, i.e., $(\sum_{i=1}^K n_i)$ active beads and $(64 - \sum_{i=1}^K n_i - n)$ inactive beads. For example, for $n = 27$ and $K = 1$, the combination $[n_1 = 27]$ results in $n_{b_0} = 10 = 64 - 27 - 27$ inactive beads and $n_{b_1} = 27$ active beads (b_1). Similarly, for combination $[n_1 = 9, n_2 = 6, n_3 = 2]$, the necklaces have $n_{b_0} = 20 = 64 - 17 - 27$, $n_{b_1} = 9$, $n_{b_2} = 6$, and $n_{b_3} = 2$ configuration.

The SageMath implementation [Sag24] (which utilizes algorithms from [Saw03]) is employed to count the number of active patterns within a specific combination. For example, given $n = 27$, $K = 6$, the combination $[n_1 = 5, n_2 = 6, n_4 = 1, n_6 = 1]$ contains $2^{42.66}$ distinct active patterns.

Table 4: Examples of combinations, necklace configurations, and counts of patterns.

K	combination	necklace	#patterns
1	$[n_1 = 27]$	$[n_{b_0} = 10, n_{b_1} = 27]$	$2^{23.17}$
2	$[n_1 = 25, n_2 = 1]$	$[n_{b_0} = 11, n_{b_1} = 25, n_{b_2} = 1]$	$2^{29.16}$
	$[n_1 = 11, n_2 = 8]$	$[n_{b_0} = 18, n_{b_1} = 11, n_{b_2} = 8]$	$2^{45.04}$
3	$[n_3 = 9]$	$[n_{b_0} = 28, n_{b_3} = 9]$	$2^{21.68}$
	$[n_1 = 9, n_2 = 6, n_3 = 2]$	$[n_{b_0} = 20, n_{b_1} = 9, n_{b_2} = 6, n_{b_3} = 2]$	$2^{48.06}$
4	$[n_1 = 1, n_2 = 2, n_3 = 2, n_4 = 4]$	$[n_{b_0} = 28, n_{b_1} = 1, n_{b_2} = 1, n_{b_3} = 2, n_{b_4} = 4]$	$2^{33.57}$
	$[n_1 = 9, n_2 = 4, n_3 = 2, n_4 = 1]$	$[n_{b_0} = 21, n_{b_1} = 9, n_{b_2} = 4, n_{b_3} = 2, n_{b_4} = 1]$	$2^{48.57}$
5	$[n_1 = 18, n_4 = 1, n_5 = 1]$	$[n_{b_0} = 17, n_{b_1} = 18, n_{b_4} = 1, n_{b_5} = 1]$	$2^{37.25}$
	$[n_1 = 8, n_2 = 4, n_3 = 2, n_5 = 1]$	$[n_{b_0} = 22, n_{b_1} = 8, n_{b_2} = 4, n_{b_3} = 2, n_{b_5} = 1]$	$2^{47.28}$

Table 4 provides example combinations and corresponding necklace configurations for $n = 27$ active Sboxes and varying K values, illustrating the diversity of pattern

subsets without requiring explicit enumeration. The size calculations serve to estimate computational complexity and guide the partitioning strategy.

3.3 Search Space Analysis and Management of Difficult Subsets

Building on the partitioning method, this section summarizes the resulting subsets of the search space and addresses strategies for managing particularly large subsets.

Overview of Search Space Structure. The total search space of active patterns is partitioned into smaller, more manageable subsets through a two-stage process: first dividing by the number of active Sboxes, then further partitioning using combinations of active groups. Figure 8 provides a visual summary of this hierarchical partitioning strategy.

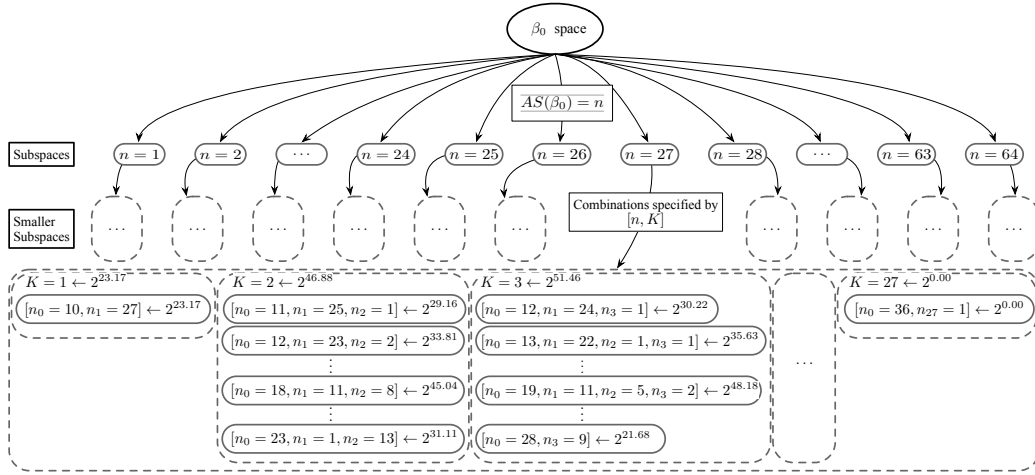


Figure 8: Two-stage partitioning of the search space.

Distribution and Size Analysis of Subspaces. The two-stage partitioning process produces subspaces with widely varying computational complexity. Table 5 provides a comprehensive view of this distribution for $20 \leq n \leq 28$, categorizing combinations by their estimated size ranges $[2^i, 2^{i+1})$. The size calculations serve as complexity indicators.

Table 5: Summary of combinations, with $[i]$ representing sizes $2^i \leq |S| < 2^{i+1}$.

n	[0,35)	[35)	[36)	[37)	[38)	[39)	[40)	[41)	[42)	[43)	[44)	[45)	[46)	[47)	[48)
20	499	22	13	20	18	16	8	13	7	9	2	-	-	-	-
21	615	30	26	17	19	18	26	8	12	7	11	3	-	-	-
22	755	31	35	35	33	19	20	24	17	12	11	7	3	-	-
23	921	36	46	37	41	42	32	24	18	25	13	14	6	-	-
24	1126	61	59	49	46	44	34	48	26	30	24	15	10	3	-
25	1350	83	82	60	55	63	60	45	51	30	28	30	11	10	-
26	1680	82	79	82	94	74	72	70	47	51	35	34	21	14	1
27	2019	110	101	112	96	113	104	87	76	51	56	29	32	18	6
28	2399	199	139	150	118	131	105	116	87	86	66	58	31	26	7

The varying sizes of subspaces present distinct computational challenges. During the SAT-based exploration process, each combination is processed sequentially. The

computational cost of analyzing a combination depends on the structural complexity of the subspace it defines, leading to three categories:

- **Small combinations** ($|S| < 2^{41}$): These subsets can be efficiently explored by the SAT solver, often requiring negligible computational time.
- **Moderate combinations** ($2^{41} \leq |S| < 2^{46}$): The time required for SAT-based analysis increases with structural complexity but remains manageable, typically ranging from a few minutes to hours.
- **Large combinations** ($|S| \geq 2^{46}$): The SAT exploration of such subspaces, particularly for $n \geq 26$, can take days or months. Fortunately, their number is small, and further partitioning can significantly reduce computational cost.

Understanding the size distribution of subsets is critical for estimating the computational complexity of the search process. Supplementary Material B provides detailed examples of the time required for the SAT solver to analyze subsets of various sizes in the context of the 2-round collision trail search model. Figure 12 presents the runtime for the SAT solver to analyze small and moderate combinations, while Figure 13 illustrates the time required to analyze subsets derived from a large combination using enhanced partitioning.

Remark 1. The above discussion about the computational cost of analyzing subspaces provides a general overview. While specific subspaces may be easier or harder to analyze depending not only on their sizes but also significantly on their structure, the overall trends described remain valid. The SAT implementation is robust across all combinations. It consistently produces results (SAT or UNSAT) within a reasonable timeframe, avoiding any scenario where it becomes stuck or fails to converge.

Summary of Large and Difficult Subsets. When $n \geq 26$, moderate and large combinations start to consume significant resources, especially the large subspaces with sizes exceeding 2^{46} , which we classify as *difficult subsets*. To better illustrate the computational challenges, Table 6 lists the number of combinations corresponding to large and difficult subspace sizes for increasing n . Each row summarizes the number of subspaces falling within specific size ranges.

Table 6: Summary of combinations with sizes $|S| \geq 2^{41}$ and $n \geq 26$.

n	[41)	[42)	[43)	[44)	[45)	[46)	[47)	[48)
26	70	47	51	35	34	21	14	1
27	87	76	51	56	29	32	18	6
28	116	87	86	66	58	31	26	7
29	156	105	103	81	62	39	33	7
30	153	173	115	103	80	51	33	9
31	198	187	157	125	84	67	33	9
32	298	228	174	135	111	77	27	5

For the very difficult subsets ($|S| \geq 2^{47}$), the number of combinations increases more gradually as n grows. However, for moderately large subspaces ($2^{42} \leq |S| < 2^{46}$), the number of combinations increases with n . Notably, when $n \geq 33$, the number of combinations and the exact sizes of each subset mirror those of complementary cases with $(64 - n)$ active Sboxes.

Supplementary Material C further categorizes large and difficult subsets by K , offering insights into the types of combinations most likely to yield challenging subspaces.

Remark 2. The search space of β_0 is closely tied to the exact differences, the size of which is computed by multiplying a factor of 2^{2n} to the subset size of patterns. Consequently, the computational complexity increases slightly as n grows, even though the number of patterns within each combination remains the same. On the other hand, when $n > 38$, the sizes of the corresponding subspaces decrease to a manageable extent.

Further Subdivision of Difficult Subsets. For difficult subsets with a large number of active patterns (e.g., $|S| > 2^{46}$), we further partition them using a two-stage approach.

In the first stage, we select a small number of the largest beads (i.e., those corresponding to active groups with the most active Sboxes). Let t_1 denote the number of selected beads, typically choosing $t_1 = 2$ or $t_1 = 3$. For example, in the necklaces shown in Figure 7, if $t_1 = 2$, we select beads b_3 and b_5 .

We then enumerate binary necklaces of $t = (64 - n)$ beads, where the selected t_1 beads are marked as 1 and the remaining beads as 0. This generates a set of necklaces that specify the positions of the t_1 largest beads.

In the second stage, for each position configuration from the first stage, we enumerate the remaining necklace structure with the t_1 largest beads fixed in their predetermined positions. This effectively reduces the problem to enumerating necklaces with $t - t_1$ variable beads.

By applying this two-stage partitioning, each difficult subset is divided into multiple smaller subsets, significantly reducing the computational complexity for SAT-based exploration. Note that the smaller subsets may overlap slightly. However, the entire combination is guaranteed to be fully covered.

3.4 SAT Encoding of Necklace-based Combinations

This section describes the SAT encoding implementation for necklace-based combinations, including methods for handling difficult subsets that require additional subdivision.

Encoding Combinations. For each combination $[n_1, n_2, \dots, n_K]$ and its associated necklace configuration $[n_{b_0}, n_{b_1}, \dots, n_{b_K}]$, the SAT implementation is constructed by adding CNF clauses that describe the constraints and relationships among the 64 *Sbox variables* (variables indicating the activeness of the Sboxes) and the *active group variables* (variables representing the presence of active groups). The encoding involves the following steps.

- *Fixing initial Sbox variables*

The first $K + 2$ Sbox variables are fixed to the value 01^K0 . This effectively anchors the position of one instance of the largest active group within the combination, breaking the translation invariance of trails and eliminates most translational redundancy⁵.

The anchoring approach significantly simplifies the SAT model implementation. Moreover, for combinations where $n_K = 1$, this step further simplifies the encoding process, making it easier to implement and enhancing computational efficiency.

- *Constraining contiguous Sbox variables*

Let K' denote the size of the largest active group in the remaining combination (i.e., excluding the initially fixed bead). Constraints are added to ensure that there are at most K' contiguous variables set to 1 in the Sbox variables

Specifically, for any $n_i = 0$ where $i \in \{1, \dots, K'\}$, constraints are added to explicitly prohibit the presence of i contiguous Sbox variables set to 1.

⁵Note that some patterns may still appear in multiple combinations. However, this does not compromise the completeness of our space partitioning approach.

- *Defining active group variables*

For each i where $n_i > 0$ (meaning there are active groups of size i in the combination), and for each of the remaining $64 - (K + 2)$ Sbox positions, we introduce a binary variable $v_{n_{ik}}$, where $k \in \{K + 1, \dots, 62\}$. This variable indicates whether an active group of size i starts at that specific Sbox position k .

If an active group of size i begins at Sbox position k , then the corresponding $v_{n_{ik}}$ variable is set to 1. Otherwise, it is set to 0.

CNF clauses are added to define the relationship between the $v_{n_{ik}}$ variables and the Sbox variables. Specifically, if $v_{n_{ik}} = 1$ at the Sbox position k , then the Sbox variables $Sbox[k + 1]$, $Sbox[k + 2]$, ..., $Sbox[k + i]$ must be set to 1 (active), and, if not at a boundary, $Sbox[k]$ and $Sbox[k + i + 1]$ should be 0 (inactive). These clauses, combined with the constraints on contiguous Sboxes, ensure that the active groups are correctly placed and do not overlap.

- *Adding cardinality constraints*

Incorporate cardinality constraints on the active group variables in the form $\sum_{K+1}^{62} v_{n_{ik}} = n_i$ to the SAT model. These constraints enforce that the total count of active groups aligns with the values specified in the combination.

The constraints and variable definitions are implemented using *truth table minimization encoding*, except for cardinality constraints which use *kmtotalizer encoding*. While the encoding process described above is conceptually straightforward, the choice of specific encoding strategies can significantly impact solver efficiency. Section 3.5.1 examines different encoding approaches and provides concrete examples of the constraint formulations.

Encoding Subdivision of Combinations. When exploring single combinations becomes computationally inefficient, we apply the two-stage subdivision approach described in Section 3.3. This method systematically partitions difficult combinations into smaller, manageable subsets by fixing the positions of the largest active groups.

The subdivision encoding extends the basic combination encoding through the following process:

Position Fixing Strategy. We select t_1 (typically 2 or 3) of the largest beads in the necklace representation and fix their positions. Each fixed position corresponds to specific Sbox variables that must be set to 1 (active) for the group's interior positions and 0 (inactive) for the boundary positions. This approach is an extension of the initial 01^K0 fixing used for the overall combination, but now applied to multiple groups simultaneously.

Encoding Implementation. For each choice of fixed positions, we identify the corresponding Sbox variables and predefine their values to reflect the presence of the t_1 active groups. This predefinition effectively reduces the remaining search space to a subset with $t - t_1$ variable beads. The encoding process for these remaining beads follows the standard combination encoding methodology described earlier.

Concrete Example. Consider the combination $[n_1 = 12, n_2 = 6, n_3 = 1]$ of size $2^{47.26}$, where $n = 27$ and $K = 3$. The corresponding necklace configuration is $[n_{b_0} = 18, n_{b_1} = 12, n_{b_2} = 6, n_{b_3} = 1]$. Setting $t_1 = 2$, we fix the positions of the single group of size 3 (i.e., the b_3 bead) and one group of size 2 (i.e., the b_2 bead).

The fixing process proceeds as follows. First, we fix the b_3 group position by setting the first $K + 2 = 5$ Sbox variables to 01110, following the standard anchoring approach. Next, we fix one b_2 group by selecting a starting position among the remaining 59 Sbox positions. For instance, placing the b_2 group at position 61 requires setting Sbox variables at positions 61 and 62 to 1, while setting the adjacent boundary positions (60 and 63) to 0, assuming no boundary conflicts.

Subdivision Results. This subdivision process generates 58 smaller subsets, each corresponding to a different placement of the b_2 group among the available positions. Each subset has significantly reduced computational complexity compared to the original combination. For each subset, the predefined Sbox variables are fixed in the SAT model, and additional constraints are applied to encode the remaining combination structure.

The complete combination is guaranteed to be exhaustively explored through this subdivision approach, though individual subsets may have slight overlaps. Runtime analysis for these smaller subsets is provided in Figure 13 in Supplementary Material B.

3.5 Encoding Strategies for Enhanced Solving Efficiency

When addressing a specific search task, multiple encoding methods can often be applied, each potentially impacting the efficiency of the SAT solver. From the user’s perspective, identifying the most effective encoding is essential for achieving optimal performance.

In this section, we discuss two encoding strategies, i.e., *capturing comprehensive relationships among variables* and *eliminating (redundant) variables*, both of which have demonstrated significant improvements in solving efficiency in our experiments.

Remark 3. The application of SAT in cryptanalysis typically involves search-related tasks, yet there is limited discussion or consensus regarding how to encode models for optimal solving efficiency. While different cryptanalytic problems often require unique approaches, and a universal encoding strategy remains unlikely, we share the observations from our implementation of SAT search models. These findings are not intended as guidelines but rather as insights that may help inspire similar efforts in other contexts.

We demonstrate the impact of encoding strategies on solver efficiency using the SAT implementation of a 2-round collision trail search with the partitioning method. Specifically, we focus on the search task modeled as $\beta_0 \xrightarrow{L} \alpha_1$, with β_0 partitioned into subsets (or combinations) defined by (n, K) , i.e., $AS(\beta_0) = n$, $AS(\alpha_1) \leq Y$.

3.5.1 Capturing Comprehensive Relationships Among Variables

In encoding search problems, a common assumption is that reducing the number of variables and CNF clauses enhances solving efficiency. However, in this collision trail search implementation, we observe that this is not always the case. Instead, capturing the relationships among variables as comprehensively as possible can significantly improve the performance of SAT solvers.

To illustrate this observation, we give two ways of encoding the combinations outlined in Section 3.4. For example, consider the combination $[n_1 = 11, n_2 = 8]$ of size $2^{45.04}$, where $n = 27$ and $K = 2$. The main CNF descriptions include:

- *Encoding constraints on contiguous Sbox variables*

In this example, since $K = 2$, and $n_2 = 8$, we have $K' = 2$. To encode the constraint that at most $K' = 2$ contiguous 1s are allowed in the Sbox variables, the CNF model of $K' + 2 = 4$ variables is constructed. Specifically, the truth table in Sbox variables $Sbox[0], Sbox[1], \dots, Sbox[K' + 2 - 1]$ is (0000, 0001, 0010, 0011, 0100, 0101, 0110, 1000, 1001, 1010, 1011, 1100, 1101), i.e., all possible values except 0111, 1110, 1111 are included, ensuring that the Sbox variable constraints are correctly encoded.

- *Defining active group variables*

The relationship between Sbox variables (begin at position k) and active group variables is encoded. For instance, for $v_{n_{1k}}$, the truth table specifies that if the Sbox variables form the pattern (0100, 0101), then $v_{n_{1k}} = 1$; otherwise, for all other patterns

(0000, 0001, 0010, 0011, 1000, 1001, 1010, 1011, 1100, 1101), $v_{n_{1k}}=0$. For $v_{n_{2k}}$, the truth table specifies that if the Sbox variables form the pattern 0110, then $v_{n_{2k}}=1$; otherwise, for all other patterns, $v_{n_{2k}}=0$. This encoding approach is applied consistently to define all active group variables $v_{n_{ik}}$, where $i \in \{1, \dots, K'\}$, $k \in \{K+1, \dots, 62\}$.

Naive Encoding Strategy. A straightforward implementation for a given combination, such as $[n_1 = 15, n_3 = 4]$, involves two steps. First, add the contiguous Sbox variable constraints illustrated earlier into the SAT model. Then, check the active groups in the combination. Since $n_1 > 0$ and $n_3 > 0$, let $k = 3$ in here, these constraints are added to the model by separately encoding $(Sbox[0], Sbox[1], \dots, Sbox[K' + 1], v_{n_{13}})$, $(Sbox[0], Sbox[1], \dots, Sbox[K' + 1], v_{n_{33}})$ to the model. While this approach seems natural, it fails to solve even small combinations ($|S| \leq 2^{30}$) within weeks.

Encoding Strategy Covering All Involved Variables. A more effective approach encodes the relationships among all variables simultaneously. This strategy combines constraints on contiguous K' -size Sbox variables, and definitions of active group variables, e.g., constructing a big truth table with all related Sbox variables and active group variables.

This comprehensive encoding uses truth tables to describe all related variables, ensuring that all relationships are explicitly defined in the CNF clauses. By encoding these constraints together, and we assume $k = 0$ in here, $(Sbox[0], Sbox[1], \dots, Sbox[K' + 1], v_{n_{10}}, \dots, v_{n_{K'0}})$, the relationships between active group variables are also captured, which are missing in the naive approach.

Comparison of Encoding Strategies. Comparing the two methods, the comprehensive encoding strategy, which captures as many relationships among variables as possible, consistently outperforms the naive approach in terms of solving efficiency. Even for large combinations, this strategy enables solvers to find solutions within a reasonable time frame.

3.5.2 Eliminating (Redundant) Variables

Before diving deeper, note that in SAT models we are discussing, additional variables that represent the activeness of Sboxes are not required for both β_0 and α_1 . For β_0 , the second row (β_0^1) inherently represents the activeness of Sboxes. Similarly, for α_1 , the sum of the second and fourth rows ($\alpha_1^1 + \alpha_1^3$) represents its Sbox activeness. Through comparison, we observe that eliminating redundant active Sbox parameters does not degrade efficiency and rather, in most cases, can improve it to some extent.

Model 1: described with $\beta_0 \xrightarrow{L} \alpha_1$	Model 2: described with β_0
Variables: α_1 variables (320) Model: α_1 constraints described on α_1 α_1 Sbox condition p_L operation $(\#\alpha_1^1 + \#\alpha_1^3) \leq Y$	Variables: $AS(\alpha_1)$ variables (64) Model: α_1 constraints described on β_0 α_1 Sbox condition p_L operation $\#AS(\alpha_1) \leq Y$

Models 1 and 2 describe two SAT encoding strategies for the search task. Model 1 includes all β_0 and α_1 , while Model 2 eliminates α_1 by transferring constraints on α_1 to β_0 .

To highlight the differences between the two encoding strategies, we omit the identical components shared by both models. These include (1) the variables of β_0 and the parameters used to encode the subsets (or combinations) in the SAT model, and (2) the constraints on β_0 , such as fixing differences to specific sets and encoding subsets (or combinations) as detailed in Section 3.4.

Model 1 employs a direct and intuitive approach to encode all conditions and constraints on both β_0 and α_1 . Specifically, the α_1 -related encoding includes constraints on α_1 (fixing differences to a specific set), the relationship between β_0 and α_1 via the p_L operation, and the cardinality constraint $AS(\alpha_1) \leq Y$, all of which are integrated into the SAT model.

Model 2 eliminates α_1 entirely, retaining only β_0 . Since L is a linear operation, all conditions and constraints on α_1 can be conveniently transformed into equivalent conditions on β_0 using the p_L operation. In this approach, an additional parameter is introduced to represent the Sbox activeness of α_1 , as α_1 is no longer explicitly included in the model.

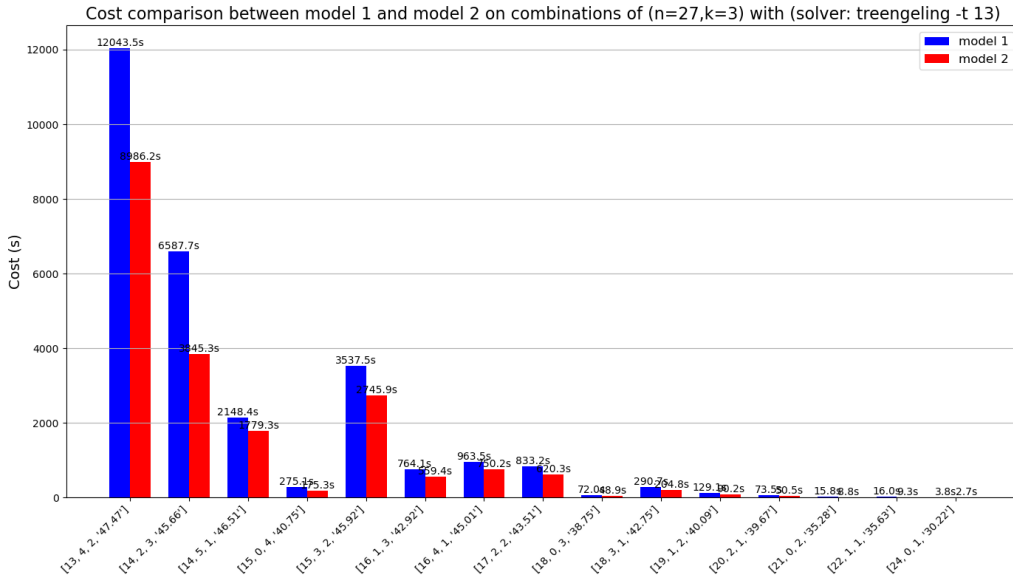


Figure 9: Runtime comparison of Model 1 and Model 2.

Comparison of Solving Efficiency. Both models are implemented in SAT for the subset collision trail search tasks. The encoding strategy in Model 2, which eliminates α_1 , outperforms the straightforward encoding strategy in Model 1. Figure 9 provides a clear comparison of their solving efficiency.

As shown, for all types of subset traversals, the variable-eliminating strategy of Model 2 consistently achieves better runtime performance. For some subsets, the runtime gain is significant, with reductions to one-third or even half of the runtime of Model 1. These results demonstrate the clear advantage of eliminating redundant variables in improving solving efficiency.

4 Differential-based Collision Attacks on Ascon-Hash256

In this section, we apply the SAT-based search space partitioning technique to find improved differential collision trails for Ascon-Hash256. We first revisit the 2-round collision attack, then demonstrate exhaustive trail search for 2 rounds and discovery of new trails for longer rounds.

4.1 2-Round Collision Attack

We analyze the time complexity of the 2-round collision attack from [YLW⁺23], focusing primarily on the complexity optimization aspects. Additional details of the attack are provided in Supplementary Material D.

The 2-round collision attack involves 4 message blocks (see Figure 2), where M_0 and M_1 have zero difference, and the collision trail is applied to the permutation processing block M_2 . The 2-round collision trail is defined as

$$\alpha_0^{256} \xrightarrow[n_c=54]{\#AS=27} \beta_0 \xrightarrow{L} \alpha_1 \xrightarrow[n_d=102]{\#AS=28} \beta_1 \xrightarrow{L} \alpha_2^{256},$$

with total weight $w = n_c + n_d = 156$, where n_c and n_d denote the weights of the connecting and colliding trails, respectively.

Fulfilling the $n_c + d$ Conditions. This 2-round collision trail (with $n_c = 54$ and $n_d = 102$) imposes $w = 156$ conditions on message pairs. Of these, $n_c + d = 82$ conditions must be satisfied by message blocks (M_0, M_1) , where d represents conditions from the colliding trail that must be fulfilled using (M_0, M_1) . The remaining $n_d - d = 74$ conditions are tested using DoFs from the M_2 block.

To find a collision, we must first obtain message pairs (M_0, M_1) that satisfy the $n_c + d = 82$ conditions (such pairs are called *valid*). Given $N_{(M_0, M_1)}$ valid pairs, each M_2 block provides $r = 64$ DoFs to test the remaining $n_d - d = 74$ conditions. Since we need 2^{74} tests in total and each valid pair contributes 2^r tests, we require $N_{(M_0, M_1)} = 2^{n_d - d - r} = 2^{74 - 64} = 2^{10}$ valid (M_0, M_1) pairs.

For generating valid (M_0, M_1) pairs, we first need to find valid M_0 messages that enable efficient generation of valid (M_0, M_1) pairs. Since each M_0 can generate 2^r potential (M_0, M_1) pairs, and each has probability $2^{-(n_c + d)} = 2^{-82}$ of being valid, we need $N_{M_0} = N_{(M_0, M_1)} \cdot 2^{(n_c + d) - r} = 2^{28}$ valid M_0 messages.

The $n_c + d$ conditions are classified into three categories based on their mathematical complexity:

- $\frac{n_c}{2}$ *simple* conditions that can be linearized into linear equations,
- $\frac{n_c}{2}$ *moderately complex* conditions requiring quadratic equation solving, and
- d *complex* conditions that necessitate exhaustive search.

Supplementary Material D.3 provides additional technical details for these condition types and their treatments.

Valid (M_0, M_1) messages are generated in three stages corresponding to these categories. In the first stage, we aim to fulfill as many of the $\frac{n_c}{2}$ simple conditions as possible by linearizing them into a system of linear equations. In the second stage, the remaining simple and moderately complex conditions are satisfied by solving quadratic equations. In the final stage, the remaining d complex conditions are addressed through exhaustive search.

To illustrate this process, we consider the value propagation in the M_1 block:

$$x_0 \xrightarrow{S} y_0^3 \xrightarrow{L} x_1^3 \xrightarrow{S} y_1^1 \xrightarrow{L} x_2^1, \quad (2)$$

where x_i^j (or y_i^j) represents the j -th row of the state in round i before (for x) or after (for y) the Sbox layer. The superscripts indicate the index of the row involved.

Consider m linearized conditions on x_2^1 (forming part of the simple conditions). Propagating backward, we identify m_0 relevant bits in y_0^3 . Of these, m_1 conditions are satisfied by controlling the inner part of S_1 , which fixes $x_{m_1}^3$ bits in x_1^3 to constants. The remaining

m_2 conditions are satisfied by setting m_2 additional bits in x_1^3 to constants using DoFs from M_1 .

With all $x_{m_1} + m_2$ bits of x_1^3 set to constants, the m simple conditions transform into linear equations. The time complexity for finding one valid M_0 message is $T_{M_0} = 2^{m_1}$. Preparing N_{M_0} valid messages requires $T_{M_0} \cdot N_{M_0} = 2^{m_1} \cdot 2^{w-2r}$ 2-round Ascon permutations.

Given a valid M_0 , there are 2^{m_2} linear systems, each containing $m + m_2$ equations. Solving these systems yields message subspaces that satisfy the simple conditions, leading to quadratic equations for the remaining $n_c - m$ conditions.

Complexity of the Connecting Phase. The time complexity of solving a system of m linear boolean equations over n variables where $m < n$, denoted by T_l , is roughly estimated as $T_l = m^2 \cdot n$ bit operations. For an overdefined quadratic system of m boolean equations over n variables, the time complexity, T_q , is estimated as

$$T_q = m^2 \cdot \binom{n}{\leq 2} + 2^{n-n_1} \cdot (n_1 + \epsilon) \cdot (n_1^2 + n_1 \cdot \epsilon + n)$$

bit operations [LMSI22], where $1 \leq n_1 < n$, and $\epsilon + n_1 = m - n_1 \cdot (n_1 - 1)/2$.

For a valid M_0 , the complexity of finding (M_0, M_1) pairs satisfying the n_c conditions, denoted by T_{M_1} , is $T_{M_1} = 2^{m_2} \cdot (T_l + T_q) \cdot 2^{-11.21}$ 2-round Ascon⁶. With those (M_0, M_1) messages, the remaining d conditions are satisfied via exhaustive search with time complexity $T_d = 2^d$.

Preparing $N_{(M_0, M_1)} = 2^{n_d-d-r}$ valid (M_0, M_1) messages requires a time complexity of

$$T_{M_1} \cdot N_{M_0} + T_d \cdot N_{(M_0, M_1)}.$$

Considering the complexity of preparing valid M_0 messages, the total complexity for generating $N_{(M_0, M_1)}$ valid (M_0, M_1) messages is

$$\begin{aligned} T_{\text{connecting}} &= T_{M_0} \cdot N_{M_0} + T_{M_1} \cdot N_{M_0} + T_d \cdot N_{(M_0, M_1)} \\ &= 2^{m_1} \cdot 2^{w-2r} + (2^{m_2} \cdot (T_l + T_q) \cdot 2^{-11.21}) \cdot 2^{w-2r} + 2^d \cdot 2^{n_d-d-r} \\ &= (2^{m_1} + 2^{m_2} \cdot (T_l + T_q) \cdot 2^{-11.21}) \cdot 2^{w-2r} + 2^{n_d-r}. \end{aligned} \quad (3)$$

Complexity of 2-Round Collision Attack. Once a valid (M_0, M_1) message satisfies the $n_c + d$ connecting conditions, the remaining $n_d - d$ colliding conditions fulfilled by M_2 can be obtained by solving a system of linear equations over M_2 . Similar to the linear equations involving M_1 , the time complexity of solving such a system is estimated as $T'_l = m^2 \cdot n$ bit operations. Thus, the complexity of the final colliding search is $T_{\text{colliding}} = N_{(M_0, M_1)} \cdot T'_l$. The total complexity of the 2-round collision attack is

$$T = T_{\text{connecting}} + T_{\text{colliding}}.$$

Given that the complexity of the colliding search is negligible compared to the connecting complexity, we focus on the connecting complexity described in Equation 3. By selecting the parameters $m = 27$, $m_1 = 31$, $m_2 = 28$, and $n_1 = 6$ (implying $\epsilon = 6$), we estimate:

$$\begin{aligned} T_l &= (27 + 28)^2 \times 64 \approx 2^{17.56}, \\ T_q &= 27^2 \times \binom{64-55}{\leq 2} + 2^3 \times 12 \times 81 \approx 2^{15.33}, \end{aligned}$$

⁶Note that each round of Ascon permutation contains $2368 = 27 \times 64 + 2 \times 320$ bit operations [DEMS21], meaning that a 2-round Ascon computation contains approximately $2^{11.21}$ bit operations. Hence, to transform the time complexity measured in bit operations to permutation units, we multiply by a factor of $2^{-11.21}$.

in bit operations, and the final collision complexity is

$$T = 2^{28} \times (2^{31} + 2^{28} \times (2^{17.56} + 2^{15.33}) \times 2^{-11.21}) + 2^{38} \approx 2^{62.63}$$

2-round Ascon.

Improved Complexity with a Modified Configuration. By modifying the configuration to $m_1 = 33$ and setting $x_{m_1} = m_2 = 26$, the complexity can be further reduced to

$$2^{28} \cdot (2^{33} + 2^{26} \cdot (T_l + T_q) \cdot 2^{-11.21}).$$

Here, $T_l + T_q$ is calculated as $T_l + T_q = (26 + 26)^2 \times 64 + 27^2 \times (11 + 11 \times 5) \approx 2^{17.75}$. Thus, the total time complexity is approximately $2^{61} + 2^{60.55} \approx 2^{61.79}$, which is slightly better than the result given in [YLW⁺23].

4.2 Collision Trail Requirements for Enhanced Attacks

4.2.1 Collision Trails for Enhanced 2-Round Collision Attacks

While the 2-round collision attack is stated practical in [YLW⁺23], obtaining a practical collision message pair remains a challenging task. To address this, we analyze the time complexity to explore possibilities for improving the attack's efficiency.

Relationship Between Collision Trails and Time Complexity. The time complexity given in Equation 3 can be simplified to highlight the dominant terms:

$$T = 2^{w-2r} \cdot (2^{m_1} + 2^{m_2} \cdot (T_l + T_q)),$$

where $T_l = (m + m_2)^2 \cdot r$, $T_q = (n_c - m)^2 \cdot \binom{r-m-m_2}{\leq 2}$, and m , m_1 , and m_2 are defined in the value propagation model in Equation 2.

Intuitively, solving linear systems is computationally more efficient than solving quadratic systems. Maximizing the number of linear equations is advantageous. For example, setting $m = \frac{n_c}{2}$, the complexity of solving T_l and T_q can be bounded as:

$$T_l + T_q \leq (64^2 \cdot 64 + (\frac{n_c}{2})^2 \cdot \binom{r - \frac{n_c}{2} - m_2}{\leq 2}) \cdot 2^{-11.2} \leq 2^7.$$

To improve the attack, the parameters must satisfy the following inequality:

$$2^{w-2r} \cdot (2^{m_1} + 2^{m_2+7}) \leq 2^{62}.^7 \quad (4)$$

By generating 2-round collision trails that meet these criteria, the complexity can be reduced.

Characteristics of Optimized Collision Trails. Using Equation 4, we can constrain parameters m_1 , m_2 , and the total weight w for better complexity. For example, if $w = 156$, then $m_1 \leq 33$ and $m_2 \leq 26$. When 33 y_0^3 bits are set constants, the maximum number of constant x_1^3 bits is 28, i.e., $x_{m_1} \leq 28$. Combining $m_2 \leq 26$, the total constant x_1^3 bits will not exceed 54, i.e., $x_{m_1} + m_2 \leq 54$.

This implies that achieving lower weights alone is insufficient; trails must also satisfy additional constraints. Table 7 summarizes configurations for w , m_1 , m_2 , and $x_{m_1} + m_2$ that lead to improved complexity.

⁷The actual value is $2^{61.79}$, but we round to 2^{62} for clarity.

Table 7: Configurations for better complexity.

w	156	157	158	159	160	161	162	163	164	165	166	167
$m_1 \leq$	33	32	31	30	29	28	27	26	25	24	23	22
$x_{m_1} \leq$	28	27	26	25	24	23	22	21	20	19	18	17
$m_2 \leq$	26	25	24	23	22	21	20	19	18	17	16	15
$x_{m_1} + m_2 \leq$	54	52	50	48	46	44	42	40	38	36	34	32

Search Objective for Collision Trails. The number of active Sboxes in the first round of the collision trail (e.g., $AS(\alpha_0) = m = \frac{n_c}{2}$) directly determines the conditions on x_2^1 . For example, when $m = 26$, the 26 conditions on x_2^1 require $x_{m_1} + m_2 \geq 32$. Simultaneously, the total weight w across the two rounds must satisfy $w \leq 166$ to ensure better complexity (see Table 7).

In the second round, the number of active Sboxes is constrained by $AS(\alpha_1) \leq \lfloor \frac{w-n_c}{3} \rfloor$, where w is determined by $x_{m_1} + m_2$. For instance, with $m = 26$, $AS(\alpha_1) \leq 38 = \lfloor \frac{166-26 \times 2}{3} \rfloor$.

Table 8 summarizes the objectives for 2-round collision trail search. In practical implementation, considering that (1) the bound $x_{m_1} + m_2$ applies to any distribution of active Sboxes in α_0 , not specifically those in a collision trail; and (2) it is unlikely that all active Sboxes in α_1 will have the smallest possible weight of 3, the objective for $AS(\alpha_1)$ is slightly reduced.

Table 8: Configurations for 2-round collision trail search.

m (or $\frac{n_c}{2}$)	25	26	27	28	29	30	31	32	33	34	35	36
$x_{m_1} + m_2 \geq$	31	32	33	34	35	36	37	38	39	40	41	42
$w \leq$	167	166	166	165	165	164	164	163	163	162	162	161
$AS(\alpha_1) \leq$	39	38	37	36	35	34	33	32	31	30	29	28
$AS^\dagger(\alpha_1) \leq$	35	34	33	33	32	32	31	30	29	28	27	26
m (or $\frac{n_c}{2}$)	37	38	39	40	41	42	43	44	45	46	47	48
$x_{m_1} + m_2 \geq$	43	44	45	46	47	48	49	50	51	52	53	54
$w \leq$	161	160	160	159	159	158	158	157	157	156	156	155
$AS(\alpha_1) \leq$	27	26	25	24	23	22	21	20	19	18	17	16
$AS^\dagger(\alpha_1) \leq$	25	24	23	22	21	20	19	18	17	16	15	14

[†] Indicates the adjusted objective for practical implementation.

4.2.2 Collision Trails for 3-/4-/5-Round Collision Attacks

For 3-/4-/5-round collision attacks, the complexity is computed by summing the costs of fulfilling both the connecting and colliding trails, i.e., $T = T_{connecting} + T_{colliding}$.

Unlike the 2-round attack, where optimization techniques are effective during the connecting phase, these techniques are not applicable to attacks spanning more than two rounds. This limitation arises because it is infeasible to formulate linear or quadratic equations to identify message pairs that satisfy the connecting trail.

Consequently, for 3-/4-/5-round collision attacks, both the connecting and colliding phases rely on exhaustive search strategies. Any attack with complexity lower than the birthday bound is considered valid.

To analyze the differential trails used in 3-/4-/5-round collision attacks, we examine the following 3-round collision trail:

$$\overbrace{\alpha_0^{256} \xrightarrow{S} \beta_0 \xrightarrow{L} \alpha_1}^{w_c} \xrightarrow{S} \beta_1 \xrightarrow{L} \alpha_2 \xrightarrow{S} \beta_2 \xrightarrow{L} \alpha_3^{256}. \quad \underbrace{\hspace{10em}}_{w_d}$$

Here, n_c represents the weight of the first round, and n_d represents the weight of the remaining rounds. Additionally, in the second round, certain conditions (denoted as d) can only be satisfied by previous message blocks. Thus, the total complexity is expressed as:

$$T = T_{connecting} + T_{colliding} = 2^{n_c+d} + 2^{n_d-d}.$$

In summary, collision trails for 3-/4-/5-round attacks must ensure that the overall time complexity remains below the birthday bound.

4.3 SAT-based Collision Trail Search with Space Partitioning

This section introduces SAT-based trail search that incorporates the partitioning strategies described in Section 3. Supplementary Material D.4 provides additional details on the general SAT model implementation and efficiency analysis for interested readers.

4.3.1 Exhaustive 2-Round Collision Trail Search

As discussed in Section 4.2.1, the 2-round collision trail search can be reduced to finding trails of the form $\beta_0 \xrightarrow{L} \alpha_1$. To optimize the time complexity of the 2-round collision attack, it is essential that the collision trails adhere to the configurations outlined in Table 8. For example, if $AS(\beta_0) = 27$, then it follows that $AS(\alpha_1) \leq 33$.

We conduct an exhaustive search over 2-round collision trails for the range $18 \leq AS(\beta_0) \leq 48$. Specifically, for the range $18 \leq AS(\beta_0) \leq 37$, we perform an exhaustive traversal of all subsets of β_0 , incorporating the partitioning strategy into the basic SAT model. The SAT implementation includes both the initial and modified models that describe these subsets. In cases where subsets are particularly difficult, further partitioning is applied to refine the search.

For $AS(\beta_0) \geq 38$, as the number of active Sboxes increases, although the combination size decreases, the number of distinct differences within these subsets grows substantially, resulting in decreased search efficiency. To address this challenge, we shift our focus from traversing β_0 to traversing α_1 .

Specifically, we fix $AS(\alpha_1) = X$ and traverse the subsets of α_1 through the partitioning strategy. For instance, when $AS(\beta_0) = 38$, we set $AS(\alpha_1) \leq 24$. This results in an upper bound for $AS(\beta_0)$, which is given by $\lfloor \frac{160-24 \times 3}{2} \rfloor = 44$. As shown in Table 9, the configurations when fixing $AS(\alpha_1)$ are obtained based on the guidelines presented in Table 8.

Table 9: Configurations for 2-round collision trail search when fixing $AS(\alpha_1)$.

$AS(\alpha_1) =$	25	24	23	22	21	20	19	18	17	16	15	14
$w \leq$	161	160	160	159	159	158	158	157	157	156	156	155
$AS(\beta_0) \leq$	43	44	45	46	48	49	50	51	53	54	55	56
$AS^\dagger(\beta_0) \leq$	36	38	39	41	42	44	45	47	48	50	51	53

[†] Indicates the adjusted objective (given by $\lfloor \frac{w-AS(\alpha_1) \times 3.5}{2} \rfloor$) for practical implementation.

For cases where $AS(\beta_0) < 18$ or $AS(\beta_0) > 48$ (or equivalently, $AS(\alpha_1) \leq 13$), the initial SAT model is sufficient for solving the problem directly, and thus, the space partitioning model is not necessary.

Findings for 2-Round Collision Trail Search. Enhanced with the partitioning technique, we conduct an exhaustive search over the space for $\beta_0 \xrightarrow{L} \alpha_1$, aiming to find trails that lead to collision attacks with better complexity.

Despite the extensive search, only trails with propagation weights greater than 156, which cannot lead to better 2-round collision attacks, are generated. Consequently, we conclude that no better 2-round collision trails exist.

The search was performed with 96 cores, using Intel® Xeon® E5-4650 v3 processors running at 2.10 GHz. The number of threads allocated for each task varied between 8 and 18. The search task was carried out over approximately ten months. This extended timeline is not indicative of low search efficiency, but rather a result of limited computational resources and an extremely large search space.

Remark 4. The bounds from our 2-round exhaustive search do not directly extend to higher rounds, primarily because the impact of the CICO problem varies across different round numbers. Indeed, despite extensive efforts utilizing various models in conjunction with the partitioning techniques, finding differential trails that lead to valid collision attacks on 3 or more rounds remains a formidable challenge. The following sections will elaborate on our search for such multi-round trails.

4.3.2 Enhanced 3-/4-/5-Round Collision Trail Search

In the context of multi-round collision trails, the best-known 4-round collision trail was proposed by Gerault *et al.* [GPT21], which involves 79 active Sboxes with a trail weight of 253. However, it has been demonstrated that this 4-round differential characteristic is invalid, as proven by Li *et al.* [LZH⁺24].

To date, no significant collision trails for 3 or 5 rounds have been proposed. In this work, we extend the enhanced 2-round trail search to longer rounds, considering up to 5 rounds. However, as the number of rounds increases, the search space grows exponentially, leading to significant search challenges. As a result, we aim to generate optimal trails within the constraints of our computational resources.

Inspired by the approach of Makarim *et al.* [MR22], we focus on the linear layer of Ascon to investigate the configurations of active Sboxes across 3-/4-/5-round collision trails. We adopt the definition of a valid configuration of active Sboxes provided in [MR22].

Definition 6 (Valid Configuration of Active Sboxes). Let $r \geq 2$ denote the number of rounds. We define a configuration $(s_0, \dots, s_{r-1})$ as valid if there exists an r -round collision trail $(\beta_0, \alpha_1, \beta_1, \alpha_{r-1})$ where the active Sboxes at each round $0, \dots, r-1$ are given by $s_0, \dots, s_{r-1}$, respectively.

To further speed up the search process, we introduce the concept of *minimum active Sboxes* after a given number of rounds. Specifically, for a given number s_0 of active Sboxes in the first linear layer at round 0, we define the minimum number of active Sboxes after the p_L transformation as $N_{min}[s_1]$. These minimum bounds are derived using the basic SAT model, as shown in Table 10.

Building on these bounds, we check the subsets of the entire search space by incorporating (1) the encoding of subsets and (2) additional cardinality constraints for each round. Specifically:

- For 3-round collision trails, we exhaustively explore the subsets of β_0 specified by (n, K) , where $n \in \{5, 6, 7\}$. The additional cardinality constraints are assigned as $(s_0 \leq 7, s_1 \leq 25, s_2 \leq 40)$.

Table 10: Differential behavior of the linear layer in the 3-/4-/5-round collision trail.

Round	s_0	$N_{min}[s_1]$	Status	Round	s_0	$N_{min}[s_1]$	Status
3	1	/	UNSAT	4/5	1	5	SAT
	2	/	UNSAT		2	7	SAT
	3	/	UNSAT		3	9	SAT
	4	/	UNSAT		4	11	SAT
	5	29	SAT		5	12	SAT
	6	25	SAT		6	13	SAT
	7	24	SAT		7	13	SAT

- For 4-round collision trails, we examine the search space α_2 specified by (n, K) , where $n \in \{27, 28, 29, 30, 31, 32\}$. The cardinality constraints are specified as $(s_0 \leq 4, s_1 \leq 13, s_2 \leq n, s_3 \leq 42)$.
- Similarly, for 5-round collision trails, we examine α_2 under the divisions of (n, K) , where $n \in \{16, 17, 18\}$. The cardinality constraints are specified as $(s_0 \leq 5, s_1 \leq 13, s_2 \leq n, s_3 \leq 35, s_4 \leq 35)$.

Findings for 3-/4-/5-Round Collision Trail Search. We find improved collision trails for 3 and 4 rounds, and discover collision trails for 5 rounds, as summarized in Table 11, where the total number of active Sboxes is denoted as $\mathcal{N}_s$.

Table 11: Best known 3-/4-/5-round collision trails

Round	Valid Configuration	$\mathcal{N}_s$	Weight	Status	Reference
3	(7, 25, 35)	67	216	Valid	Table 21
4	(3, 10, 41, 43)	97	295	Valid	[DEMS19]
	(2, 9, 30, 38)	79	253	Invalid	[GPT21]
	(3, 11, 31, 33)	78	250	Valid	Table 23
5	(1, 5, 18, 33, 35)	92	298	Invalid	Table 25
	(3, 11, 17, 31, 34)	96	310	Unknown	Table 26

To validate these collision trails, we applied the verification algorithm by Li et al. [LZH⁺24]. The 3-round and 4-round trails are confirmed valid, enabling 3-/4-round SFS collision attacks, with concrete message pairs in Table 22 and Table 24 (provided in Supplementary Material D.5), respectively.

For the 5-round trails, the one with weight 298 is invalid, while the trail with weight 310 remains unverified due to SAT solver limitations, preventing any 5-round SFS collision attack.

5 MitM-based Collision and Preimage Attacks

Having demonstrated its effectiveness in collision trail search, we now apply the space partitioning technique to find improved MitM structures for both collision and preimage attacks. While the search targets differ, the strategy remains effective for managing the large search spaces involved. The SAT implementation follows similar principles. For brevity, we focus on presenting the improved attacks and their complexity analysis.

5.1 Improved MitM Collision Attack on 3-Round Ascon-Hash256

We adopt the MitM collision attack framework introduced in Section 2.2. As shown in Figure 3, the MitM structure is applied to the final block M_2 . This structure identifies messages (M_0, M_1, M_2) that contain d_M fixed zeros in the inner part of the output. Preparing $2^{\frac{h-d_M}{2}}$ such messages statistically ensures a collision.

With the partitioning technique, we conduct an exhaustive search over all possible 3-round MitM collision structures under the half-rate model (i.e., $rate = 32$). This complete exploration identifies the optimal structure that provides the best complexity improvement, characterized by parameters $d_R = d_B = 24$ and $d_M = 32^8$.

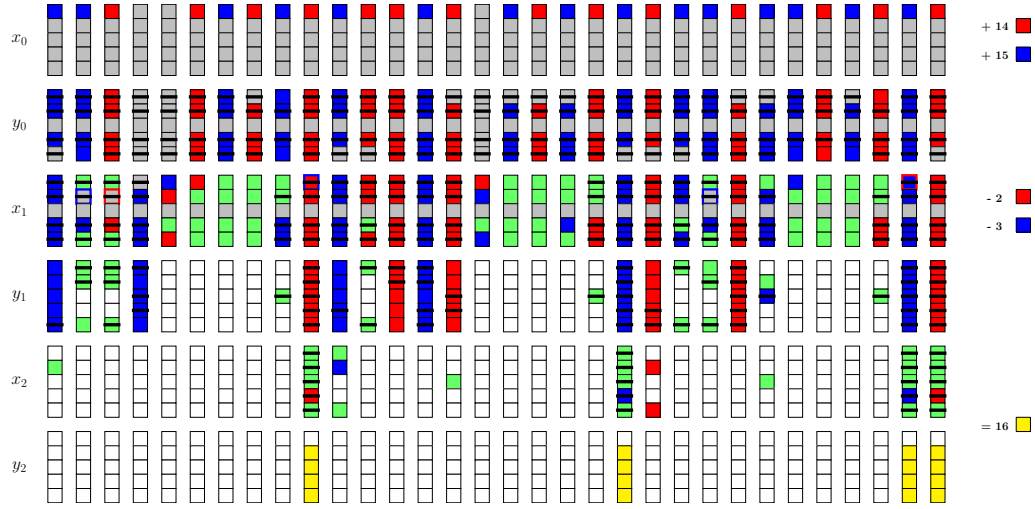


Figure 10: The 3-round collision MitM structure with $d_R = d_B = 24$, $d_M = 32$, and $i_c = 30$.

The key parameters of the collision MitM structure (Figure 10) are defined as follows. The preimage MitM structure (Figure 11) follows similar parameter conventions.

- f_R, f_B, f_C : Initial Degrees of Freedom (DoFs) for ■, ■, and ■ bits. In this structure, $f_R = 28$, $f_B = 30$, and $f_C = 6$. Denote each of the 2^{f_R} ■ values by v_R and each of the 2^{f_B} ■ values by v_B .
- C_R (respectively C_B): The DoFs consumed when propagating ■ and ■ bits respectively. These represent the number of bits that must be fixed to constants to prevent further propagation. The corresponding constant values, i.e., ■/■ and ■/■, are denoted by c_R and c_B .
- d_M : The number of matching bits in the output state fixed to specific values (i.e., zeros for collision attacks, target values for preimage attacks).
- d_R, d_B : The remaining DoFs for ■ and ■ bits at the matching point.
- i_C : The number of conditions imposed on the inner part of the intermediate output $H(M_0, M_1)$.

⁸Figure 10 displays the structure using half-rate model parameters ($d_R = d_B = 12$, $d_M = 16$). Throughout our analysis, we use doubled values ($d_R = d_B = 24$, $d_M = 32$) to reflect full-rate complexity.

Algorithm 1: MitM collision attack on 3-round Ascon-Hash256

```

1 for  $2^N \cdot 2^{i_C}$  values of  $(M_0, M_1)$  do
2   if one valid  $(M_0, M_1)$  message is found then
3     With the 256-bit inner part of  $H(M_0, M_1)$ , compute and store the related  $\blacksquare$ 
       bits in state  $y_0$  and  $x_1$ .
4     for all  $2^{f_C}$   $\blacksquare$  values in  $x_0$  do
5       Propagate the  $\blacksquare$  value one round, to compute, store and update related
        $\blacksquare$  bits in  $y_0$  and  $x_1$ .
6       /* Store the  $\blacksquare$  values in  $2^{C_R} = 2^4$  tables, each containing
        $2^{f_R - C_R} = 2^{24}$  distinct entries. */
7       for all  $2^{f_R}$   $\blacksquare$  values  $v_R$  in  $x_0$  do
8         Propagate  $v_R$  2 rounds to compute the  $\blacksquare$  bits and red-dependent
         part of the  $\blacksquare$  bits in  $y_0$ ,  $x_1$ , and  $y_1$ , and the matching value  $m_R$ .
9         /* For those  $c_B$ -dependent bits, compute and store for
         all possible  $c_B$  values. */
10        Get the  $C_R$ -bit  $\blacksquare/\blacksquare$  constant  $c_R$ . Store the entry  $(v_R, m_R)$  in  $U_{c_R}$ .
11      end
12      /* Store the  $\blacksquare$  values in  $2^{C_B} = 2^6$  tables, each containing
       $2^{f_B - C_B} = 2^{24}$  distinct entries. */
13      for all  $2^{f_B}$   $\blacksquare$  values  $v_B$  in  $x_0$  do
14        Propagate  $v_B$  2 rounds to compute the  $\blacksquare$  bits and blue-dependent
        part of the  $\blacksquare$  bits in  $y_0$ ,  $x_1$ , and  $y_1$ , and the matching value  $m_B$ .
15        /* For those  $c_R$ -dependent bits, compute and store for
        all possible  $c_R$  values. */
16        Get the  $C_B$ -bit  $\blacksquare/\blacksquare$  constant  $c_B$ . Store the entry  $(v_B, m_B)$  in  $U_{c_B}$ .
17      end
18      for each  $c_R \in \{0, \dots, 2^{C_R} - 1\}$  and the table  $U_{c_R}$  do
19        for each  $c_B \in \{0, \dots, 2^{C_B} - 1\}$  and the table  $U_{c_B}$  do
20          /* Given specific  $c_R$  and  $c_B$ , retrieve corresponding
          matching values for related  $v_B$  and  $v_R$ . */
21          for each entry  $(v_B, m_B)$  in current  $U_{c_B}$  do
22            If  $m_B$  passes the 4-bit filter, store the  $v_B$  in table  $L_1$  indexed
            by the remaining 24-bit matching value.
23          end
24          for each entry  $(v_R, m_R)$  in current  $U_{c_R}$  do
25            if  $m_R$  passes the 4-bit filter then
26              Retrieve the matching  $v_B$  from  $L_1$ . Concatenate  $v_R$ ,  $v_B$ ,
              and the  $\blacksquare$  value to obtain a candidate  $v$ .
              Compute the 256-bit inner part (denoted by  $c_M$ ) from  $v$ .
              Store  $(M_0, M_1, M_2)$  in hash table  $L$  indexed by  $c_M$ .
              /*  $M_2$  is the outer part of  $v \oplus H(M_0, M_1)$ . */
              if  $|L| = 2^{\frac{h-d_M}{2}} = 2^{112}$  then
                return a collision  $(M_0, M_1, M_2)$  and  $(M'_0, M'_1, M'_2)$ .
              end
            end
          end
        end
      end
    end
  end

```

Algorithm 1 presents the complete MitM collision attack procedure. To evaluate the complexity, we adopt the 3-round Ascon permutation as the computational unit, consistent with the differential-based collision attacks.

For precise complexity assessment, we count the number of involved Sbox output bits rather than simply counting Sboxes. A 3-round Ascon permutation involves 192 Sbox operations (64 Sboxes per round), corresponding to 960 output bits. In Figure 10, relevant bits are marked with black horizontal lines through their centers; this excludes $\blacksquare$ bits, which are computed solely from inner part constants.

The attack complexity analysis is organized into three main parts.

Part 1: Preparing Valid (M_0, M_1) Messages. Messages (M_0, M_1) satisfying the i_C conditions are considered valid. Given that $2^{\frac{h-d_M}{2}}$ messages (M_0, M_1, M_2) are required ($h = 256$), and each valid (M_0, M_1) generates 2^{r-d_M} messages, we need to prepare $2^N = 2^{\frac{h-d_M}{2} - (r-d_M)} = 2^{r+\frac{d_M}{2}} = 2^{80}$ valid (M_0, M_1) messages (since $h = 4r$).

Hence, the time complexity for preparing 2^N valid (M_0, M_1) messages is $2^N \cdot 2^{i_C} \times 2 = 2^{80+30+1} = 2^{111}$ 3-round Ascon computations.

Part 2: Constructing Matching Messages. Using Lines 3-23 in Algorithm 1, we construct $2^{\frac{h-d_M}{2}} = 2^N \cdot 2^{r-d_M} = 2^{112}$ matching messages from the valid (M_0, M_1) pairs. Each matching message v has $d_M = 32$ zero bits at the specified matching positions.

The time complexity of computing the $\blacksquare$ bits in states y_0 and x_1 shown in Lines 3-5 is

$$2^N \cdot \left(\frac{100}{960} + 2^{f_C} \cdot \frac{18}{960} \right) = 2^{80} \cdot \frac{100 + 2^6 \times 18}{960} \approx 2^{80.38}$$

3-round Ascon. The term $\frac{100}{960}$ is because, given the 256-bit inner part of x_0 , computing the related $\blacksquare$ bits in y_0 and x_1 involves 100 Sbox output bits. Similarly, the term $2^{f_C} \cdot \frac{18}{960}$ accounts for the 2^{f_C} freely chosen $\blacksquare$ values in x_0 , where computing the related $\blacksquare$ bits involves 18 Sbox output bits.

The time complexity of computing the red matching values $m_{\mathcal{R}}$ in state x_2 (Lines 7-10), which requires propagating $\blacksquare$ values through intermediate states and computing related $\blacksquare$ bits and red-dependent part of $\blacksquare$ bits in states y_0 , x_1 , and y_1 , is

$$2^N \cdot 2^{f_R} \cdot \left(\frac{94}{960} + 2^{f_C} \cdot \frac{70}{960} \right) = 2^{80+28} \cdot \frac{94 + 2^6 \times 70}{960} \approx 2^{110.25}$$

3-round Ascon. For each of the 2^{28} $\blacksquare$ values, 94 Sbox output bits are involved in the computation of related bits in y_0 and x_1 , giving a cost of $\frac{94}{960}$. To further obtain the related bits in y_1 and the red matching values, 70 additional Sbox output bits must be computed. Since the 2^{f_C} freely chosen $\blacksquare$ values affect a portion of these Sbox operations, this contributes an extra complexity of $2^{f_C} \cdot \frac{70}{960}$.

Remark 5. In this process, a key computational challenge arises when computing $m_{\mathcal{R}}$ and $m_{\mathcal{B}}$, i.e., the matching conditions consist of red and blue terms, where some red terms depend on blue constants and some blue terms depend on red constants. Specifically, among the 32 matching conditions, 12 red terms depend on $\blacksquare/\blacksquare$ constants, while 12 blue terms depend on $\blacksquare/\blacksquare$ constants. To handle this efficiently, we employ a pre-computation strategy that stores the computation results for both possible values (0 and 1) of each dependent constant. Since computing the result for one constant value automatically yields the result for the alternative value, this approach incurs no additional computational overhead while enabling direct lookup during the matching phase. Supplementary Material E provides additional analysis of the matching conditions and implementation details of this pre-computation strategy.

Similarly, the time complexity of computing blue matching values m_B (Lines 12-15), including the related $\blacksquare$ and blue-dependent $\blacksquare$ bits in states y_0 , x_1 , and y_1 , is

$$2^N \cdot 2^{f_B} \cdot \left(\frac{94}{960} + 2^{f_C} \cdot \frac{68}{960} \right) = 2^{80+30} \cdot \frac{94 + 2^6 \times 68}{960} \approx 2^{112.21}$$

3-round Ascon. For each of the 2^{30} $\blacksquare$ values, 94 Sbox output bits are involved in the computation of related bits in y_0 and x_1 , giving a cost of $\frac{94}{960}$. To further obtain the related bits in y_1 , 68 additional Sbox output bits must be computed. Since the $\blacksquare$ values in x_0 affect a portion of these Sbox operations, this contributes an extra complexity of $2^{f_C} \cdot \frac{68}{960}$.

When all red and blue matching values are prepared, the time complexity of constructing the matching messages in Lines 16-23 is

$$2^N \cdot 2^{f_C} \cdot 2^{C_R} \cdot 2^{C_B} \cdot \left(2^{d_B} \cdot \frac{1}{192} + 2^{d_R-4} \cdot \frac{1}{192} \right) = 2^{80+6+4+6} \cdot \frac{2^{24} + 2^{20}}{192} \approx 2^{112.50}$$

3-round Ascon. The algorithm iterates through all combinations of constants $c_R \in \{0, \dots, 2^{C_R} - 1\}$ and $c_B \in \{0, \dots, 2^{C_B} - 1\}$, processing the corresponding tables U_{c_R} and U_{c_B} . As detailed in Supplementary Material E, the 32 matching conditions are structured as follows: 4 conditions involve only red values, 4 involve only blue values, and 24 combine both red and blue terms. This creates 4-bit filters for both red and blue entries.

For each table U_{c_B} containing 2^{d_B} blue entries, only 2^{d_B-4} entries pass the 4-bit blue filter. These surviving blue values v_B are stored in hash table L_1 , indexed by their 24-bit matching values (derived from the 24 mixed red-blue conditions after applying the 4-bit filter). Following the complexity estimation method from [DZQ⁺24], we estimate the cost of inserting each value as one Sbox computation, yielding a cost of $2^{d_B} \cdot \frac{1}{192}$.

Similarly, for each red entry (v_R, m_R) in table U_{c_R} , the 4-bit red filter reduces 2^{d_R} entries to 2^{d_R-4} survivors. Each surviving red value v_R retrieves its matching blue value v_B from L_1 and constructs the complete candidate message v by concatenating v_R , v_B , and the gray values. The cost for this step is estimated as $2^{d_R-4} \cdot \frac{1}{192}$.

This process constructs $2^{f_C} \cdot 2^{C_R} \cdot 2^{C_B} \cdot \frac{2^{d_R-4} \cdot 2^{d_B-4}}{2^{d_M-8}} = 2^{r-d_M}$ matching messages for each valid (M_0, M_1) pair. In total, $2^N \cdot 2^{r-d_M} = 2^{112}$ candidate messages are prepared.

Part 3: Collision Detection. Propagate each candidate matching message v through the 3-round Ascon to compute its 256-bit inner part, denoted by c_M . We then store the complete message tuple (M_0, M_1, M_2) in hash table L , indexed by c_M , where M_2 represents the outer part of $v \oplus H(M_0, M_1)$.

After storing approximately $2^{\frac{h-d_M}{2}} = 2^{112}$ such entries, we expect to find a collision pair when two distinct message tuples map to the same c_M . The time complexity of this phase is 2^{112} 3-round Ascon operations.

Complexity Summary. The total time complexity is

$$2^{111} + 2^{80.38} + 2^{110.25} + 2^{112.21} + 2^{112.50} + 2^{112} \approx 2^{114.13}$$

3-round Ascon, improving upon the previous best result of $2^{116.74}$.

For memory complexity, preparing tables for each valid (M_0, M_1) requires storing $2^{\max(f_R-C_R, f_B-C_B)}$ entries (i.e., 2^{24} in this case), which is negligible compared to storing the 2^{112} message tuples (M_0, M_1, M_2) for the collision search. Thus, the memory complexity is 2^{112} .

5.2 Improved MitM Preimage Attack on 3-Round Ascon-Hash256

Following the two-phase MitM preimage attack framework described in Section 2.2, we employ the MitM procedure in Phase I to find partial preimages (see Figure 4), while Phase II completes the full preimage attack.

We apply the same partitioning approach to discover a preimage MitM structure shown in Figure 11. This structure achieves improved complexity for 3-round Ascon-Hash256 with parameters $f_C = 0$, $f_R = 224$, $f_B = 32$, and $d_R = d_B = d_M = 32$.

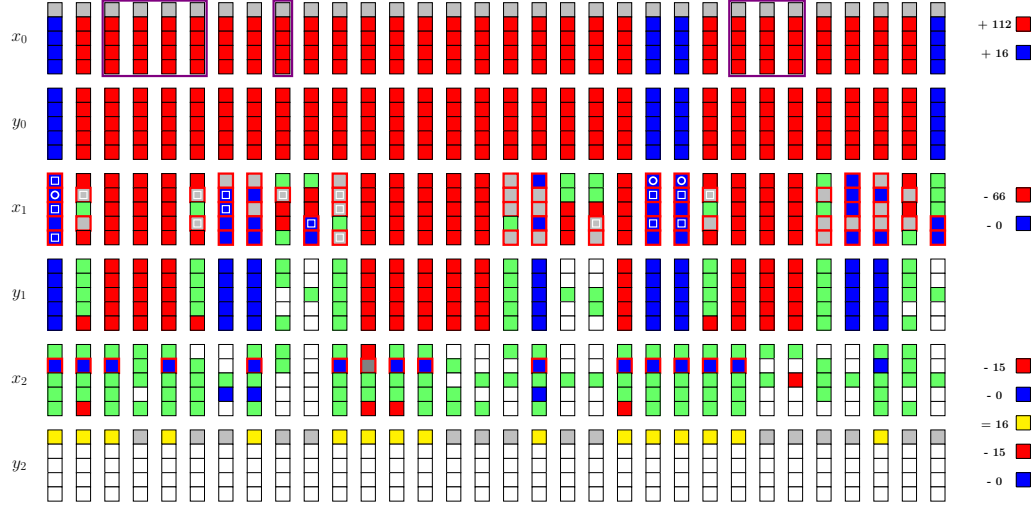


Figure 11: The 3-round preimage MitM structure with $d_R = d_B = d_M = 32$.

In this MitM structure, blue bits maintain their full DoFs throughout propagation, while red bits undergo significant reduction. Specifically, the first p_L operation consumes $C_R^1 = 132$ red DoFs, the second p_L consumes $C_R^2 = 28$ red DoFs, and achieving the desired matching consumes an additional $C_R^M = 32$ red DoFs by converting $\blacksquare$ bits (of $x_2^{i,0} + x_2^{i,2} + x_2^{i,4}$) to $\blacksquare$ bits, where $x_2^{i,j}$ represents the j -th bit of the i -th Sbox in state x_2 (these Sboxes correspond to the matching points).

To effectively apply the MitM approach, we must organize the $2^{f_R} = 2^{224}$ $\blacksquare$ values into structured tables. Ideally, each table would contain $2^{f_R - C_R^1 - C_R^2 - C_R^M} = 2^{d_R} = 2^{32}$ values sharing identical $\blacksquare/\blacksquare$ constants across all 3-round operations (totaling $C_R^1 + C_R^2 + C_R^M$ bits). Unlike previous work [DZQ⁺24] where 2^{160} red values could be processed directly, our structure's 2^{224} red values would exceed the exhaustive search complexity, requiring a more efficient approach.

Precomputation for MitM Preimage Attack. We develop Algorithm 2 to efficiently organize the large space of $\blacksquare$ values. The algorithm divides the initial $f_R = 224$ $\blacksquare$ bits into two parts: part A (64 bits) and part B (160 bits)⁹, as shown in Figure 11 where bits in part A are enclosed by purple frames.

Following the first p_L operation, the resulting $C_R^1 = 132$ $\blacksquare/\blacksquare$ constant bits in x_1 can be classified into three distinct categories:

- $X = 38$ $\blacksquare/\blacksquare$ constant bits jointly influenced by both parts A and B, marked with white-bordered squares inside the bits;

⁹Here, part A and B can be chosen freely. We use this specific instance to show that the time complexity to prepare the values for MitM attack can be reduced to not dominate the overall time complexity, even though the initial DoFs exceed the size of the messages required to do the attack.

Algorithm 2: Organizing red values for MitM preimage attack

```

1 /* Process the 64-bit values in part A */
2 for all  $2^{64}$  values of the ■ bits in part A do
3   Apply  $p_S$  operation and compute values of the  $(Y + X)$ -bit ■/■ constants in  $x_1$ .
4   Store the 64-bit values into  $2^Y$  piles  $P_A$  based on values of the  $Y$ -bit constants,
      each containing  $2^X$  tables  $S_A$ .
5   /* Each table  $S_A$  stores  $2^{64-Y-X} = 2^{20}$  values sharing the same
       $(Y + X)$ -bit constants. */
  end
6 /* Process the 160-bit values in part B (store only  $2^{24}$  piles) */
7 for all  $2^{160}$  values of the ■ bits in part B do
8   Apply  $p_S$  operation and compute values of the  $(Z + X)$ -bit ■/■ constants in  $x_1$ .
9   Organize the 160-bit values into  $2^Z$  piles  $P_B$  based on values of the  $Z$ -bit
      constants, each containing  $2^X$  tables  $S_B$ .
10  /* Each table  $S_B$  stores  $2^{160-Z-X} = 2^{34}$  values sharing the same
       $(Z + X)$ -bit constant. */
11  Among the  $2^Z$  piles, store only  $N_{P_B} = 2^{192-d_M-(f_R-C_R^1)-(X+Y)} = 2^{24}$  piles for
      further processing
  end
12 /* Construct final subsets by merging parts A and B */
13 for each of the  $N_{P_B}$  stored piles  $P_B$  do
14   for each possible  $Y$ -bit constant value  $y \in \{0, 1, \dots, 2^Y - 1\}$  do
15     for each possible  $X$ -bit constant value  $x \in \{0, 1, \dots, 2^X - 1\}$  do
16       for each  $S_B$  of  $X$ -bit  $x_B \in \{0, 1, \dots, 2^X - 1\}$  in current  $P_B$  do
17         Find matching  $S_A$  with  $X$ -bit constant  $x_A = x \oplus x_B$  from pile  $P_A(y)$ .
18         Concatenate values from  $S_B$  and  $S_A$  to form a new table  $S_R$  of
            224-bit ■ values.
19         /* Each  $S_R$  stores  $2^{34} \cdot 2^{20} \cdot 2^{38} = 2^{92}$  values. */
       end
     end
  end
end
end
20 return  $N = N_{P_B} \cdot 2^Y \cdot 2^X = 2^{68}$  tables, each storing  $2^{92}$  red values.
```

- $Y = 6$   constant bits determined solely by part A, marked with white-bordered circles inside the bits;
- $Z = 88$   constant bits determined solely by part B, with no special marking.

For processing part A (Lines 2-5), the time complexity is $2^{64} \cdot \frac{16}{192} \approx 2^{60.42}$ 3-round Ascon, where $\frac{16}{192}$ accounts for the 16 involved Sboxes. The memory complexity is 2^{64} .

For part B (Lines 7-11), examining all 2^{160} values requires $2^{160} \cdot \frac{40}{192} \approx 2^{157.74}$ 3-round Ascon. We store only $N_{PB} = 2^{24}$ piles, requiring $2^{24+38+34} = 2^{96}$ memory units.

The final merging phase (Lines 13-20) constructs $N = 2^{68}$ tables by concatenating values from parts A and B. Following the complexity estimation in [DZQ⁺24], each table insertion costs one Sbox operation, yielding $N \cdot 2^{92} \cdot \frac{1}{192} \approx 2^{152.42}$ 3-round Ascon with 2^{160} memory.

The total precomputation complexity is $2^{60.42} + 2^{157.74} + 2^{152.42} \approx 2^{157.74}$ 3-round Ascon and 2^{160} memory.

Algorithm 2 produces $N = 2^{68}$ initial tables, each containing  values that share identical 132-bit   constants in x_1 . To enable the MitM attack, these tables must be further subdivided based on additional constant bits that emerge during the remaining operations. Specifically, the   constant bits from the second round ($C_R^2 = 28$ bits) and those required for achieving the matching conditions ($C_R^M = 32$ bits) allow each initial table to be divided into 2^{60} smaller tables, each containing exactly $2^{d_R} = 2^{32}$ red values.

Attack Algorithm and Complexity Analysis. Utilizing the tables from Algorithm 2, we present an improved preimage attack on 3-round Ascon-Hash256. Algorithm 3 outlines Phase I. Recall that $H_1 \parallel H_2 \parallel H_3 \parallel H_4$ represents the 256-bit hash value, with the MitM structure applied during the squeezing of H_2 and matching occurring in specific bits of H_2 .

The time and memory complexity are illustrated below.

Part 1: Processing Initial Tables (Lines 2-9). The time complexity for processing all red values in the initial tables is

$$2^{68} \cdot 2^{92} \cdot \left(\frac{56 \times 5 + (18 \times 5 + 10) + 32}{960} + \frac{1}{192} \right) \approx 2^{160} \cdot 2^{-1.20} = 2^{158.80}$$

3-round Ascon.

For each of the 2^{68} initial tables containing 2^{92} red values, we propagate each value through the 3-round structure and organize them into smaller tables. The propagation computation involves 56 fully active Sboxes in the first round, 18 fully active Sboxes plus 10 additional output bits in the second round, and 32 output bits for the matching point. The $\frac{1}{192}$ term represents the cost of inserting each red value into its corresponding hash table.

Part 2: Constructing Candidate Matching Messages (Lines 12-19). The time complexity for processing blue values and constructing candidate matching messages is

$$2^{32} \cdot \frac{8}{192} + 2^{68} \cdot 2^{32} \cdot \frac{154}{960} + 2^{68} \cdot 2^{60} \cdot 2^{32} \cdot \frac{32}{960} \approx 2^{160} \cdot 2^{-4.92} = 2^{155.08}$$

3-round Ascon.

In the first round, blue values propagate independently of any red constants, requiring only $2^{32} \cdot \frac{8}{192}$ to compute and store the y_0 values for all blue inputs. In the second round, the 132-bit red constants begin affecting Sbox operations. For each of the 2^{68} different red constant combinations (corresponding to initial tables), we compute and store the propagation of 2^{32} blue values through x_1 and y_1 states, involving 154 Sbox output bits. These intermediate values are stored and reused across the 2^{60} hash tables U_{c_R} that share

Algorithm 3: MitM preimage attack on 3-round Ascon-Hash256 (Phase I)

```

1 Prepare  $N = 2^{68}$  initial tables using Algorithm 2.
2 for each initial table  $S_R$  do
3   /* Further subdivide each initial table into  $2^{C_R^2+C_R^M} = 2^{60}$  smaller
   tables, each storing  $2^{92-C_R^2-C_R^M} = 2^{32}$  values. */
4   for all  $2^{92}$  red values  $v_R$  in the current table  $S_R$  do
5     Propagate  $v_R$  forward through the rounds, computing all relevant
       intermediate state bits (i.e., ■ bits and red-dependent part of ■ bits) of  $y_0$ ,
        $x_1$ ,  $y_1$ , and  $x_2$ .
6     Determine value of the  $C_R^2$ -bit ■/■ constant in  $x_2$ .
7     Determine value of the  $C_R^M$ -bit ■/■ constant for the matching.
8     Calculate the  $d_M = 32$ -bit matching value  $m_R$ .
9     Store  $v_R$  in the smaller table  $U_{c_R}$ , indexed by the matching value  $m_R$ .
10    /*  $c_R$  represents the concatenation of all  $C_R^1$ -bit,  $C_R^2$ -bit,
       and  $C_R^M$ -bit ■/■ constants. */
    end
11   /* Generate matched messages and check preimage for each  $U_{c_R}$ . */
12   for each smaller table  $U_{c_R}$  do
13     for all  $2^{f_B} = 2^{32}$  blue values  $v_B$  in  $x_0$  do
14       /* Compute the blue value in the matching point. */
15       Given the specific  $c_R$  constant, propagate  $v_B$  forward, computing all
       relevant intermediate state bits (i.e., ■ bits and blue-dependent part of
       ■ bits) of  $y_0$ ,  $x_1$ ,  $y_1$ , and  $x_2$ .
16       Calculate the 32-bit matching value  $m_B$ .
17       /* Identify the red value that matches the blue value. */
18       Compute the red matching value  $m_R = m_B \oplus H_2^M$ , where  $H_2^M$  is the
       32-bit target digest value at the matching point in  $H_2$ .
19       Retrieve the corresponding entry  $(v_R, m_R)$  from  $U_{c_R}$  and construct
       candidate message  $v = v_R \parallel v_B$  that matches the target 32-bit digest
       (of  $H_2$ ) at the matching point.
20       /* Verify if candidate is a valid preimage. */
21       Combining  $H_1$  with  $v$ , compute complete output through 3-round Ascon
       to obtain  $H_2$ .
22       if  $H_2$  matches target value then
23         Compute  $H_3$  based on the current state.
24         if  $H_3$  matches target value then
25           Compute  $H_4$  based on the current state.
26           if  $H_4$  matches target value then
27             return  $v$  as preimage of  $H_2 \parallel H_3 \parallel H_4$ .
           end
         end
       end
     end
   end
  end

```

the same 132-bit red constant. Finally, the computation to derive the 32-bit blue matching values from these propagated values requires $2^{68} \cdot 2^{60} \cdot 2^{32} \cdot \frac{32}{960}$ operations.

As soon as the blue matching values are obtained, we can directly retrieve the matching red value and construct candidate matching messages by combining red and blue values.

Part 3: Verification (Lines 21-27). The verification step to check whether candidates are valid preimages has a complexity of

$$2^{68} \cdot 2^{60} \cdot 2^{32} + 2 \times 2^{128} + 3 \times 2^{64} \approx 2^{160}$$

3-round Ascon.

The process involves three sequential stages. First, each candidate undergoes a 3-round Ascon to check if the remaining 32 bits of H_2 match the target value. Only 2^{128} candidates are expected to pass. These surviving candidates then undergo a second 3-round Ascon to verify the 64-bit H_3 , with 2^{64} candidates expected to pass. Finally, the remaining candidates undergo a third 3-round Ascon to verify the 64-bit H_4 .

The complexity formula reflects this cascading verification, i.e., 2^{160} for the first stage, 2×2^{128} for the second stage, and 3×2^{64} for the third stage.

Complexity Summary. Combining all parts with the precomputation complexity from Algorithm 2, the total time complexity is

$$2^{157.74} + 2^{158.80} + 2^{155.08} + 2^{160} \approx 2^{160.75}$$

3-round Ascon.

The memory complexity includes several components. The precomputation storage requires 2^{160} red values from Algorithm 2. During the attack, the algorithm processes one initial table at a time, with runtime storage including 2^{92} red values temporarily stored in smaller tables U_{c_R} and intermediate values for $O(2^{32})$ blue value computations. The overall memory complexity is 2^{160} red values.

Once we obtain a preimage for $H_2 \parallel H_3 \parallel H_4$ in Phase I, Phase II employs a generalized collision framework with complexity $\mathcal{O}(2^{128})$ to find a complete preimage for the full hash output $H_1 \parallel H_2 \parallel H_3 \parallel H_4$.

In summary, the improved MitM preimage attack on 3-round Ascon-Hash256 achieves time complexity $2^{160.75}$ and memory complexity 2^{160} , improving upon the previous best complexity of $2^{162.80}$.

6 Conclusion

In this work, we introduce a SAT-based space partitioning technique that addresses fundamental challenges in cryptanalytic search problems. By modeling large search spaces as combinatorial necklace generation tasks, our approach enables systematic exploration while maintaining completeness guarantees. The technique divides vast search spaces into manageable subsets with precise tracking of explored regions, overcoming the limitations of existing SAT-based methods that often become stuck in indefinite solving processes.

We demonstrated the technique's effectiveness through comprehensive cryptanalysis of Ascon-Hash256, targeting both differential-based collision attacks and Meet-in-the-Middle (MitM) structure search. For differential-based collision attacks, we conducted the first exhaustive search of 2-round collision trails, providing a theoretical guarantee that no collision trail with weight less than 156 exists. Through parameter optimization, we achieved an improved 2-round collision attack with complexity $2^{61.79}$. The technique also discovered improved SFS collision trails for 3 and 4 rounds, with the 4-round trail improving from weight 295 to 250.

The space partitioning technique proved equally effective for MitM structure search, yielding improvements in attacks on 3-round **Ascon-Hash256**. We reduced the collision attack complexity from $2^{116.74}$ to $2^{114.13}$ with memory complexity 2^{112} (improved from 2^{116}), and the preimage attack complexity from $2^{162.80}$ to $2^{160.75}$ with memory complexity 2^{160} (improved from 2^{162}).

Acknowledgment

The authors would like to thank the reviewers for their insightful comments and detailed suggestions which helped improve the manuscript significantly. This work is supported by the National Key Research and Development Program of China (2022YFB2701900), the National Cryptologic Science Foundation of China (2025NCSF01012), the National Natural Science Foundation of China (62032014, 62372262, 62532013, 62502493, 62502494), the Fundamental Research Funds for the Central Universities, and the Fellowship Funds from China Postdoctoral Science Foundation (2024M753187, 2024M763221).

References

- [BDG⁺21] Zhenzhen Bao, Xiaoyang Dong, Jian Guo, Zheng Li, Danping Shi, Siwei Sun, and Xiaoyun Wang. Automatic search of meet-in-the-middle preimage attacks on AES-like hashing. In *EUROCRYPT 2021*, volume 12696 of *Lecture Notes in Computer Science*, pages 771–804. Springer, 2021.
- [BDPVA07] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Sponge functions. In *ECRYPT hash workshop*, volume 2007, 2007.
- [BHMSV06] Robert K Brayton, Gary D Hachtel, Curt McMullen, and Alberto Sangiovanni-Vincentelli. *Espresso: a logic minimizer*. University of Texas, 2006. Online manual.
- [BKK24] Seungjun Baek, Giyoon Kim, and Jongsung Kim. Preimage attacks on reduced-round ascon-xof. *Des. Codes Cryptogr.*, 92(8):2197–2217, 2024.
- [BM22] Emanuele Bellini and Rusydi H. Makarim. Functional cryptanalysis: application to reduced-round Xoodoo. *IACR Cryptol.*, 2022:134, 2022. <https://eprint.iacr.org/2022/134>.
- [BPF⁺23] Emanuele Bellini, Alessandro De Piccoli, Mattia Formenti, David Gérard, Paul Huynh, Simone Pelizzola, Sergio Polese, and Andrea Visconti. Differential cryptanalysis with sat, smt, milp, and CP: a detailed comparison for bit-oriented primitives. In *CANS*, volume 14342 of *Lecture Notes in Computer Science*, pages 268–292. Springer, 2023.
- [DDL25] Mathieu Degré, Patrick Derbez, Lucie Lahaye, and André Schrottenloher. New models for the cryptanalysis of ascon. *Designs, Codes and Cryptography*, pages 1–18, 2025.
- [DEMS15] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schläpfer. Cryptanalysis of Ascon. In *CT-RSA*, volume 9048 of *Lecture Notes in Computer Science*, pages 371–387. Springer, 2015.
- [DEMS19] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schläpfer. Preliminary analysis of Ascon-Xof and Ascon-Hash (version 0.1), 2019.

- [DEMS21] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schl  ffer. Ascon v1.2: lightweight authenticated encryption and hashing. *J. Cryptol.*, 34(3):33, 2021.
- [DHAK18] Joan Daemen, Seth Hoffert, Gilles Van Assche, and Ronny Van Keer. The design of xoodoo and xooff. *IACR Trans. Symmetric Cryptol.*, 2018(4):1–38, 2018.
- [DHS⁺21] Xiaoyang Dong, Jialiang Hua, Siwei Sun, Zheng Li, Xiaoyun Wang, and Lei Hu. Meet-in-the-middle attacks revisited: Key-recovery, collision, and preimage attacks. In *CRYPTO 2021*, volume 12827 of *Lecture Notes in Computer Science*, pages 278–308. Springer, 2021.
- [DLL62] Martin Davis, George Logemann, and Donald Loveland. A machine program for theorem-proving. *Communications of the ACM*, 5(7):394–397, 1962.
- [DP60] Martin Davis and Hilary Putnam. A computing procedure for quantification theory. *Journal of the ACM (JACM)*, 7(3):201–215, 1960.
- [DZQ⁺24] Xiaoyang Dong, Boxin Zhao, Lingyue Qin, Qingliang Hou, Shun Zhang, and Xiaoyun Wang. Generic MitM attack frameworks on sponge constructions. In *CRYPTO (4)*, volume 14923 of *Lecture Notes in Computer Science*, pages 3–37. Springer, 2024.
- [EME22] Johannes Erlacher, Florian Mendel, and Maria Eichlseder. Bounds for the security of Ascon against differential and linear cryptanalysis. *IACR Trans. Symmetric Cryptol.*, 2022(1):64–87, 2022.
- [FLYS25] Qinggan Fu, Ye Luo, Qianqian Yang, and Ling Song. Preimage and collision attacks on reduced ascon using algebraic strategies. *Cybersecur.*, 8(1):34, 2025.
- [GLST22] Jian Guo, Guozhen Liu, Ling Song, and Yi Tu. Exploring SAT for cryptanalysis: (quantum) collision attacks against 6-round SHA-3. In *ASIACRYPT (3)*, volume 13793 of *Lecture Notes in Computer Science*, pages 645–674. Springer, 2022.
- [GPT21] David G  rault, Thomas Peyrin, and Quan Quan Tan. Exploring differential-based distinguishers and forgeries for Ascon. *IACR Trans. Symmetric Cryptol.*, 2021(3):102–136, 2021.
- [HMMD22] Solane El Hirsch, Silvia Mella, Alireza Mehrdad, and Joan Daemen. Improved differential and linear trail bounds for ASCON. *IACR Trans. Symmetric Cryptol.*, 2022(4):145–178, 2022.
- [IMM18] Alexey Ignatiev, Antonio Morgado, and Joao Marques-Silva. PySAT: a Python toolkit for prototyping with SAT oracles. In *SAT*, pages 428–437, 2018.
- [LHC⁺23] Huina Li, Le He, Shiyao Chen, Jian Guo, and Weidong Qiu. Automatic preimage attack framework on Ascon using a linearize-and-guess approach. *IACR Trans. Symmetric Cryptol.*, 2023(3):74–100, 2023.
- [LLW24] Yingxin Li, Fukang Liu, and Gaoli Wang. New records in collision attacks on SHA-2. In *EUROCRYPT (1)*, volume 14651 of *Lecture Notes in Computer Science*, pages 158–186. Springer, 2024.

- [LMSI22] Fukang Liu, Willi Meier, Santanu Sarkar, and Takanori Isobe. New low-memory algebraic attacks on lowmc in the picnic setting. *IACR Trans. Symmetric Cryptol.*, 2022(3):102–122, 2022.
- [LZ22] Huina Li and Haochen Zhang. Xoodoo SAT solver. <https://github.com/HuinaLi/XoodooSat>, 2022.
- [LZH⁺24] Huina Li, Haochen Zhang, Kai Hu, Guozhen Liu, and Weidong Qiu. Algsat - a SAT method for verification of differential trails from an algebraic perspective. In *ACISP (1)*, volume 14895 of *Lecture Notes in Computer Science*, pages 450–471. Springer, 2024.
- [MDA17] Silvia Mella, Joan Daemen, and Gilles Van Assche. New techniques for trail bounds and application to differential trails in Keccak. *IACR Trans. Symmetric Cryptol.*, 2017(1):329–357, 2017.
- [MDA23] Silvia Mella, Joan Daemen, and Gilles Van Assche. Tighter trail bounds for Xoodoo. *IACR Trans. Symmetric Cryptol.*, 2023(4):187–214, 2023.
- [MP13] Nicky Mouha and Bart Preneel. Towards finding optimal differential characteristics for ARX: application to Salsa20. *IACR Cryptol.*, 2013:328, 2013. <https://eprint.iacr.org/2013/328>.
- [MR22] Rusydi H. Makarim and Raghvendra Rohit. Towards tight differential bounds of Ascon a hybrid usage of SMT and MILP. *IACR Trans. Symmetric Cryptol.*, 2022(3):303–340, 2022.
- [MSP⁺17] Aaron Meurer, Christopher P. Smith, Mateusz Paprocki, Ondřej Čertík, Sergey B. Kirpichev, Matthew Rocklin, Amit Kumar, Sergiu Ivanov, Jason K. Moore, Sartaj Singh, Thilina Rathnayake, Sean Vig, Brian E. Granger, Richard P. Muller, Francesco Bonazzi, Harsh Gupta, Shivam Vats, Fredrik Johansson, Fabian Pedregosa, Matthew J. Curry, Andy R. Terrel, Štěpán Roučka, Ashutosh Saboo, Isuru Fernando, Sumith Kulal, Robert Cimrman, and Anthony Scopatz. Sympy: symbolic computing in Python. *PeerJ Computer Science*, 3:e103, January 2017.
- [NIS18] NIST. The NIST lightweight cryptography project, 2018. <https://csrc.nist.gov/Projects/lightweight-cryptography>.
- [OLH⁺13] Toru Ogawa, Yangyang Liu, Ryuzo Hasegawa, Miyuki Koshimura, and Hiroshi Fujita. Modulo based CNF encoding of cardinality constraints and its application to MaxSAT solvers. In *ICTAI*, pages 9–17. IEEE Computer Society, 2013.
- [Sag24] Sage. Necklaces. <https://doc.sagemath.org/html/en/reference/combinat/sage/combinat/necklace.html>, 2024.
- [Saw03] Joe Sawada. A fast algorithm to generate necklaces with fixed content. *Theoretical Computer Science*, 301(1-3):477–489, 2003.
- [Son13] Sontrak. Logic friday. <https://web.archive.org/web/20131216141103/http://sontrak.com/index.html>, 2013.
- [STMC⁺25] Meltem Sönmez Turan, Kerry McKay, Donghoon Chang, Jinkeon Kang, and John Kelsey. Ascon-based lightweight cryptography standards for constrained devices: Authenticated encryption, hash, and extendable output functions. Technical Report NIST Special Publication 800-232, National Institute of Standards and Technology, 2025.

- [SWW21] Ling Sun, Wei Wang, and Meiqin Wang. Accelerating the search of differential and linear characteristics with the SAT method. *IACR Trans. Symmetric Cryptol.*, 2021(1):269–315, 2021.
- [YLW⁺23] Xiaorui Yu, Fukang Liu, Gaoli Wang, Siwei Sun, and Willi Meier. A closer look at the S-box: deeper analysis of round-reduced Ascon-Hash. In *International Conference on Selected Areas in Cryptography*, pages 22–42. Springer, 2023.

A Basics of SAT Encoding

Boolean Satisfiability Problem (SAT). The Boolean Satisfiability Problem (SAT) [MP13] involves determining whether a Boolean formula $f(x_1, x_2, \dots, x_n)$ has an assignment of variables that makes it true. If such an assignment exists, the formula is **SAT**; otherwise, it is **UNSAT**.

Modern SAT solvers primarily rely on the DPLL (Davis-Putnam-Logemann-Loveland) algorithm [DP60, DLL62] and its enhanced variant, the CDCL (Conflict-Driven Clause Learning) algorithm. These solvers accept formulas in Conjunctive Normal Form (CNF), where a formula is expressed as a conjunction ($\wedge$) of clauses, each being a disjunction ($\vee$) of literals. A *literal* is either a variable or its negation.

SAT Encoding for Differential Trail Search. To model the differential trail search as a SAT problem, the differential propagation rules of the cryptographic primitive, along with related constraints, are encoded into CNF clauses. The solutions to the SAT problem correspond to the differential trails.

Encoding linear operations. For the differential trail search of Ascon permutation, linear operations include XOR and cyclic shifts. While cyclic shifts require no encoding, XOR operations, particularly for $n > 2$, demand careful consideration. Below are two common encoding methods:

- **Direct CNF Encoding.** This method avoids introducing additional variables but generates 2^n clauses for an XOR involving n variables. For instance, for $n = 2$, the CNF representation is

$$(\neg\beta_1 \vee \beta_2 \vee \alpha) \wedge (\beta_1 \vee \neg\beta_2 \vee \alpha) \wedge (\beta_1 \vee \beta_2 \vee \neg\alpha) \wedge (\neg\beta_1 \vee \neg\beta_2 \vee \neg\alpha).$$

- **Pairwise XOR Encoding with Intermediate Variables.** This method performs a sequence of XOR operations with $n - 2$ intermediate variables m_i . This results in $4(n - 1)$ clauses. For example,

$$m_1 = \beta_1 \oplus \beta_2, \quad m_i = \beta_{i+1} \oplus m_{i-1} \quad \text{where } 2 \leq i \leq n - 2, \quad \alpha = \beta_n \oplus m_{n-2}.$$

Encoding nonlinear operations. Nonlinear operations, such as Sbox transformations or set constraints (e.g., $\beta_0 \in \{0, 9, 11, 24, 26\}$ for a 2-round Ascon collision trail), are encoded using *truth-table minimization*. Tools like Sympy [MSP⁺17], Logic Friday [Son13], and Espresso [BHMSV06] can generate optimized CNFs from truth tables. This approach has been effectively applied in prior work [LHC⁺23].

Encoding the objective function. Differential trail searches generally involves *cardinality constraints* on the number of active Sboxes (or weight) across r rounds. These constraints are expressed as

$$\sum_{j=0}^{r-1} \sum_{i=0}^{n-1} s_j^i \cdot T,$$

where s_j^i indicates whether the i -th Sbox in round j is active or not. The symbol $(\cdot)$ represents relational operators such as $\geq$, $\leq$ or $=$. Various encoding methods for cardinality constraints are implemented in PySAT [IMM18].

Erlacher *et al.* [EME22] evaluated different encoding methods and observed that their performance depends on the satisfiability and complexity of the problem. In this work, we also evaluate several encodings and adopt the *kmtotalizer* method [OLH⁺13], which exhibits better performance for most of our SAT instances.

B Examples of Traversal Time for Combinations of Different Sizes

This section presents examples of the time consumed by the SAT solver to traverse subsets defined by combinations of various sizes in the context of the 2-round collision trail search model. All experiments were conducted on a machine equipped with an Intel(R) Xeon(R) CPU E5-4650 v3 @ 2.10GHz (12 cores), 65 GB of RAM. The SAT solver used for these experiments was Treengeling, configured to utilize 12 threads for parallel computation.

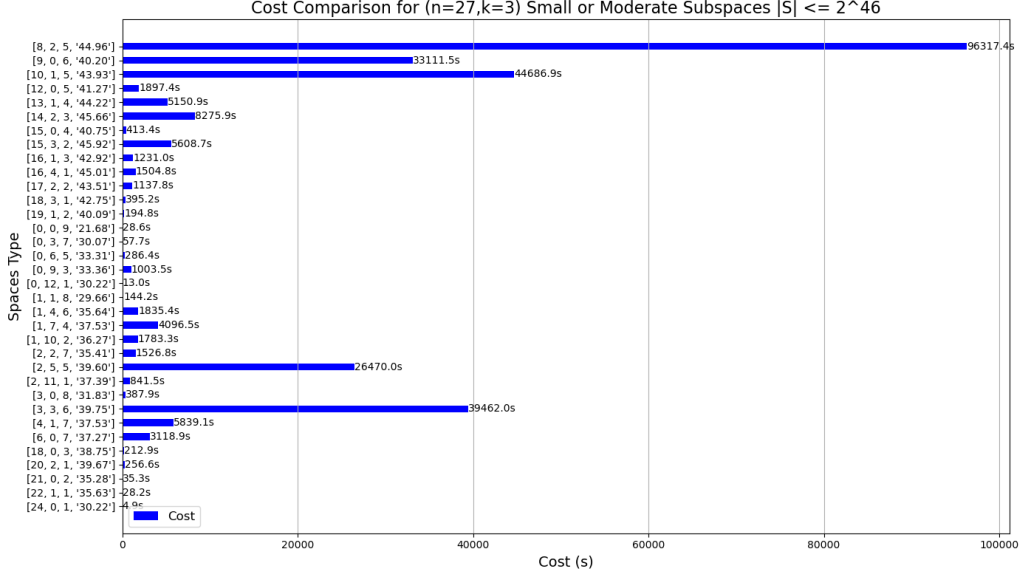


Figure 12: Traversal time for small and moderate-sized combinations.

Figure 12 presents the runtime comparison for small and moderate-sized combinations, while Figure 13 illustrates the time required to traverse smaller subsets derived from large combinations using the enhanced partitioning method.

The results demonstrate to some extent that traversal times increase with the size of the subsets and are significantly affected by the structure and size of the combinations. For small and moderate combinations, traversal times are manageable (Figure 12). However, for large combinations, breaking the subset into smaller partitions greatly improves efficiency, as shown in Figure 13.

C Large and Difficult Combinations Categorized by K

When n is fixed, multiple combinations correspond to different K , implying that combinations (or subspaces) can be grouped based on K . To provide a clearer perspective, Table 12 lists the total sizes ($\geq 2^{45}$) of combinations for $20 \leq n \leq 28$ and $2 \leq K \leq 9$.

The columns labeled from $K = 2$ to $K = 9$ show the total sizes of all subspaces defined by the corresponding (n, K) . For example, for $(n = 27, K = 3)$, there are 61 combinations, and the summed size of these combinations is $2^{51.46}$, meaning there are such number of active patterns follow the specific (n, K) . The column $|S|$ presents the total size of the subspace for a given n , summing over all the combinations defined by all K . For larger n (e.g., 26, 27, 28), the total size $|S|$ increases significantly, and certain combinations (particularly those with $K = 3, 4, 5$) tend to dominates the subset.

To highlight the most prominent subspaces, Table 13 summarizes the sizes of dominant

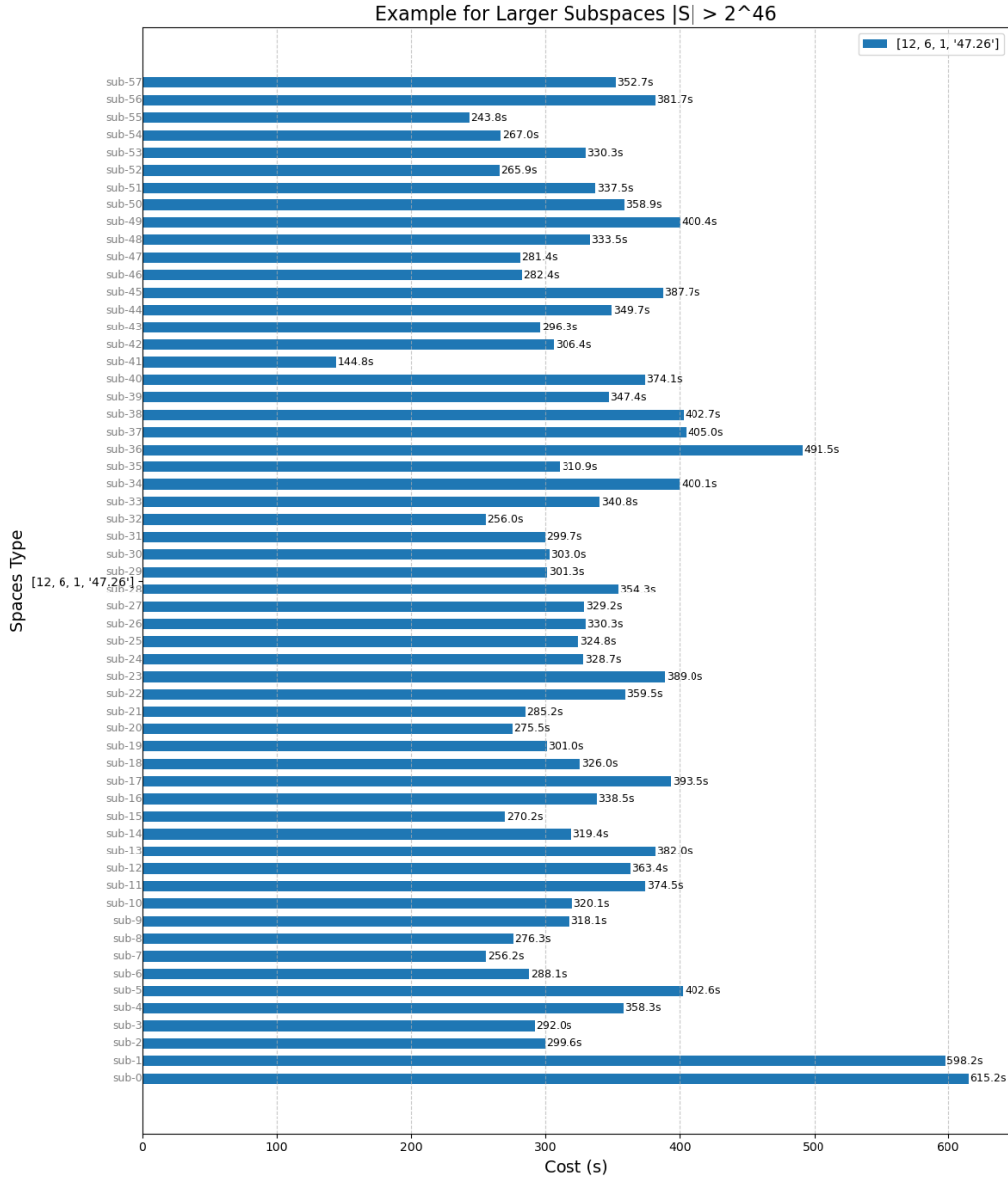


Figure 13: Traversal time for a large combination with subset further partitioned for efficient search.

Table 12: Total sizes of subspace formed by combinations for varying n and K .

n	$K = 2$	$K = 3$	$K = 4$	$K = 5$	$K = 6$	$K = 7$	$K = 8$	$K = 9$	$ S $
20	$2^{45.72}$	$2^{47.07}$	$2^{46.02}$	-	-	-	-	-	$2^{48.12}$
21	$2^{46.36}$	$2^{48.11}$	$2^{47.27}$	$2^{45.68}$	-	-	-	-	$2^{49.19}$
22	$2^{46.86}$	$2^{49.01}$	$2^{48.39}$	$2^{46.93}$	$2^{45.23}$	-	-	-	$2^{50.16}$
23	$2^{47.19}$	$2^{49.77}$	$2^{49.39}$	$2^{48.07}$	$2^{46.46}$	-	-	-	$2^{51.03}$
24	$2^{47.37}$	$2^{50.40}$	$2^{50.26}$	$2^{49.08}$	$2^{47.57}$	$2^{45.95}$	-	-	$2^{51.80}$
25	$2^{47.38}$	$2^{50.89}$	$2^{51.01}$	$2^{49.97}$	$2^{48.56}$	$2^{47.03}$	$2^{45.42}$	-	$2^{52.48}$
26	$2^{47.22}$	$2^{51.24}$	$2^{51.64}$	$2^{50.74}$	$2^{49.44}$	$2^{47.99}$	$2^{46.46}$	-	$2^{53.06}$
27	$2^{46.88}$	$2^{51.46}$	$2^{52.14}$	$2^{51.40}$	$2^{50.20}$	$2^{48.83}$	$2^{47.38}$	$2^{45.88}$	$2^{53.55}$
28	$2^{46.36}$	$2^{51.53}$	$2^{52.51}$	$2^{51.95}$	$2^{50.85}$	$2^{49.56}$	$2^{48.19}$	$2^{46.76}$	$2^{53.96}$

subspaces categorized by K . This provides an overview of space division and illustrates the challenges in traversing these subspaces.

Table 13: Summary of dominant subspaces over the whole search space categorized by K .

n	$K = 3$	$K = 4$	$K = 5$	$K = 6$	$K = 7$	$K = 8$	$K = 9$	$K = 10$	$ S $
26	$2^{51.24}$	$2^{51.64}$	$2^{50.74}$	$2^{49.44}$	$2^{47.99}$	$2^{46.46}$	-	-	$2^{53.06}$
27	$2^{51.46}$	$2^{52.14}$	$2^{51.40}$	$2^{50.20}$	$2^{48.83}$	$2^{47.38}$	-	-	$2^{53.55}$
28	$2^{51.53}$	$2^{52.51}$	$2^{51.95}$	$2^{50.85}$	$2^{49.56}$	$2^{48.19}$	$2^{46.76}$	-	$2^{53.96}$
29	$2^{51.47}$	$2^{52.76}$	$2^{52.37}$	$2^{51.39}$	$2^{50.18}$	$2^{48.88}$	$2^{47.52}$	$2^{46.12}$	$2^{54.27}$
30	$2^{51.26}$	$2^{52.88}$	$2^{52.68}$	$2^{51.81}$	$2^{50.69}$	$2^{49.46}$	$2^{48.17}$	$2^{46.84}$	$2^{54.49}$
31	$2^{50.89}$	$2^{52.87}$	$2^{52.88}$	$2^{52.13}$	$2^{51.09}$	$2^{49.93}$	$2^{48.71}$	$2^{47.45}$	$2^{54.62}$
32	$2^{50.38}$	$2^{52.72}$	$2^{52.95}$	$2^{52.33}$	$2^{51.38}$	$2^{50.30}$	$2^{49.14}$	$2^{47.94}$	$2^{54.67}$

Additional details for large and difficult subspaces within $26 \leq n \leq 32$ are also presented in this section. For each specific n , and for the K values ranging from 2 to 9, Table 14, Table 15, Table 16, Table 17, Table 18, Table 19 and Table 20 separately list the number of subspaces within specific size range. Those tables help understand the computational effort required to traverse these subspaces.

Table 14: The number of difficult subsets for $\#AS(\beta_0) = 26$

K	[40,41)	[41,42)	[42,43)	[43,44)	[44,45)	[45,46)	[46,47)	[47,48)	[48,49)
2	-	1	1	1	2	2	-	-	-
3	2	4	1	6	4	5	4	8	1
4	8	12	8	10	8	12	9	6	-
5	9	16	16	12	10	10	8	-	-
6	20	15	8	13	9	5	-	-	-
7	15	10	8	8	2	-	-	-	-
8	9	8	5	1	-	-	-	-	-
9	7	4	-	-	-	-	-	-	-

D Details of 2-Round Collision Attack

Aside from the Free-Start (FS) and Semi-Free-Start (SFS) collision attacks, the most effective collision attack on Ascon-Hash256 covers 2 rounds [YLW⁺23], which is based on the differential trail of probability 2^{-156} . The authors claim that the attack has a time complexity of $2^{62.6}$ 2-round Ascon permutations, which they describe as practical. In this section, we will review the collision attack.

D.1 Differential Properties of Ascon Sbox

The algebraic degree of the 5-bit Ascon Sbox is 2, which significantly influences its differential properties. Due to this low algebraic degree, the probability of each differential entry (α, β) in the differential distribution table (DDT) follows a pattern of 2^{-m} , where m can be 2, 3, or 4. This patterns also determines the number of output differences β for a

Table 15: The number of difficult subsets for $\#AS(\beta_0) = 27$

K	[40,41)	[41,42)	[42,43)	[43,44)	[44,45)	[45,46)	[46,47)	[47,48)	[48,49)
2	2	-	2	-	3	1	-	-	-
3	4	5	4	2	6	4	6	6	3
4	12	10	12	10	11	8	14	7	3
5	19	19	15	16	16	8	10	5	-
6	20	20	18	11	14	7	2	-	-
7	22	13	15	8	6	1	-	-	-
8	12	11	8	4	-	-	-	-	-
9	8	8	2	-	-	-	-	-	-

Table 16: The number of difficult subsets for $\#AS(\beta_0) = 28$

K	[40,41)	[41,42)	[42,43)	[43,44)	[44,45)	[45,46)	[46,47)	[47,48)	[48,49)
2	1	1	1	1	3	-	-	-	-
3	3	3	4	5	5	6	6	7	2
4	4	18	10	13	11	15	10	11	5
5	20	20	19	19	18	20	9	8	-
6	23	27	15	24	16	12	6	-	-
7	17	21	18	14	10	5	-	-	-
8	16	12	13	8	3	-	-	-	-
9	10	9	7	2	-	-	-	-	-

given input difference α . Specifically, each input indifference α has exactly 2^m compatible output differences β .

Furthermore, the low algebraic degree means that for any compatible differential (α, β) , the set of input messages x that adhere to the differential entry will form an affine subspace of size 2^{5-m} . In other words, those input messages are solutions to a system of m linear equations defined by the differential (α, β) .

These differential properties can be illustrated using the following *value-differential relation*, where α^j , β^j , and x^j denote the j -th bit (with j starting from 0) of the 5-bit input value or input and output differences of the Ascon Sbox.

$$\begin{aligned}
\beta^0 &= x^1(\alpha^0 + \alpha^2 + \alpha^4) + (x^0 + x^2 + x^4)\alpha^1 + \alpha^0 + \alpha^1 + \alpha^2 + \alpha^3 \\
&\quad + \alpha^1(\alpha^0 + \alpha^2 + \alpha^4) \\
\beta^1 &= x^1(\alpha^2 + \alpha^3) + x^2(\alpha^1 + \alpha^3) + x^3(\alpha^1 + \alpha^2) + \alpha^0 + \alpha^1 + \alpha^2 + \alpha^3 \\
&\quad + \alpha^4 + \alpha^1\alpha^2 + \alpha^1\alpha^3 + \alpha^2\alpha^3 \\
\beta^2 &= x^3\alpha^4 + x^4\alpha^3 + \alpha^1 + \alpha^2 + \alpha^4 + \alpha^3\alpha^4 \\
\beta^3 &= x^0(\alpha^3 + \alpha^4) + (x^3 + x^4)\alpha^0 + \alpha^0 + \alpha^1 + \alpha^2 + \alpha^3 + \alpha^4 + \alpha^0(\alpha^3 + \alpha^4) \\
\beta^4 &= x^1(\alpha^0 + \alpha^4) + (x^0 + x^4)\alpha^1 + \alpha^1 + \alpha^3 + \alpha^4 + \alpha^1(\alpha^0 + \alpha^4)
\end{aligned} \tag{5}$$

For example, consider the following 2-round collision trail model, which includes both

Table 17: The number of difficult subsets for $\#AS(\beta_0) = 29$

K	[40,41)	[41,42)	[42,43)	[43,44)	[44,45)	[45,46)	[46,47)	[47,48)	[48,49)
2	-	2	1	3	-	-	-	-	-
3	7	4	2	7	4	7	6	8	1
4	15	10	16	12	14	15	12	12	6
5	16	29	21	22	22	22	10	13	-
6	31	36	20	24	23	11	11	-	-
7	26	26	25	17	13	7	-	-	-
8	21	22	11	14	5	-	-	-	-
9	13	17	7	4	-	-	-	-	-

Table 18: The number of difficult subsets for $\#AS(\beta_0) = 30$

K	[40,41)	[41,42)	[42,43)	[43,44)	[44,45)	[45,46)	[46,47)	[47,48)	[48,49)
2	2	-	3	-	-	-	-	-	-
3	2	5	6	2	8	8	6	7	-
4	9	15	20	8	17	20	13	12	7
5	29	30	24	27	29	20	18	12	2
6	38	30	42	28	23	21	12	2	-
7	43	29	26	30	16	10	2	-	-
8	30	18	28	13	10	1	-	-	-
9	25	14	16	7	-	-	-	-	-

differential and value propagations:

$$\begin{aligned} \text{differential propagation : } \alpha_0^{256} &\xrightarrow{S} \beta_0 \xrightarrow{L} \alpha_1 \xrightarrow{S} \beta_1 \xrightarrow{L} \alpha_2^{256}, \\ \text{value propagation : } x_0 &\xrightarrow{S} y_0 \xrightarrow{L} x_1 \xrightarrow{S} y_1 \xrightarrow{L} x_2. \end{aligned}$$

Suppose each Sbox is indexed by i , where $i \in \{0, \dots, 63\}$. For the k -th round, the j -th bit of the input, the input difference, and the output difference of the i -th Sbox are denoted by $x_k^{i,j}$, $\alpha_k^{i,j}$, and $\beta_k^{i,j}$, respectively, where $k \in \{0, 1, 2\}$, $i \in \{0, \dots, 63\}$, and $j \in \{0, 1, 2, 3, 4\}$.

In this differential collision attack model, the first round differential for the i -th Sbox, represented by (α_0^i, β_0^i) , follows the pattern (10000, **0**), where $\beta_0^i \in \{9, 11, 24, 26\}$. The input values x_0^i of the Sbox corresponds to the solution space of the following linear Equation 6, which is derived from Equation 5.

$$(1, \beta_0^i) \Rightarrow \begin{cases} x_0^{i,1} = \beta_0^{i,0} + 1 \\ x_0^{i,3} + x_0^{i,4} = \beta_0^{i,3} + 1 \end{cases} \quad (6)$$

Additionally, the second round differential for the i -th Sbox, denoted by (α_1^i, β_1^i) follows the pattern (**0**, 10000), with $\alpha_1^i \in \{3, 9, 18, 25\}$. For each such differential, the corresponding system of equations on the input messages is listed in the following

Table 19: The number of difficult subsets for $\#AS(\beta_0) = 31$

K	[40,41)	[41,42)	[42,43)	[43,44)	[44,45)	[45,46)	[46,47)	[47,48)	[48,49)
2	2	3	-	-	-	-	-	-	-
3	2	5	7	3	9	5	8	4	-
4	13	15	17	16	21	14	16	13	6
5	28	27	32	36	23	27	24	11	3
6	40	32	47	38	30	22	15	5	-
7	41	44	31	28	27	13	4	-	-
8	41	25	28	22	12	3	-	-	-
9	26	26	15	11	3	-	-	-	-

Table 20: The number of difficult subsets for $\#AS(\beta_0) = 32$

K	[40,41)	[41,42)	[42,43)	[43,44)	[44,45)	[45,46)	[46,47)	[47,48)	[48,49)
2	2	-	-	-	-	-	-	-	-
3	6	6	5	8	5	8	8	-	-
4	18	20	10	23	18	19	19	12	3
5	32	42	37	28	35	31	25	12	2
6	45	55	49	41	32	29	20	3	-
7	48	60	51	29	28	18	5	-	-
8	41	47	35	28	12	6	-	-	-
9	28	34	24	12	5	-	-	-	-

Equation 7.

$$\begin{array}{c|c|c|c}
\boxed{(3, 16)} & \boxed{(9, 16)} & \boxed{(18, 16)} & \boxed{(25, 16)} \\
x_1^{i,0} = 0 & x_1^{i,0} = 0 & x_1^{i,0} + x_1^{i,3} = 1 & x_1^{i,0} + x_1^{i,4} = 0 \\
\boxed{x_1^{i,2}} = 0 & x_1^{i,1} + x_1^{i,4} = 1 & x_1^{i,1} = 1 & \boxed{x_1^{i,2}} = 1 \\
x_1^{i,3} + x_1^{i,4} = 0 & \boxed{x_1^{i,2}} = 0 & \boxed{x_1^{i,2}} = 1 & x_1^{i,3} = 0 \\
& x_1^{i,3} = 0 & x_1^{i,4} = 0 &
\end{array} \quad (7)$$

From these differential-value relations of the 2-round collision trail, we can observe that the input values of the first round and the second round active Sbox must satisfy certain conditions. For instance, $x_0^{i,1}$ bit of the first round active Sbox and $x_1^{i,2}$ bit of the second round active Sbox are fixed to specific constants.

Note that, like other χ Sboxes with an algebraic degree 2, there are numerous other cryptographic properties of the Ascon Sbox that are not covered here, as they are not utilized in this work.

D.2 The Connecting and Colliding Strategy to Mount Collision Attack

According to the attack framework introduced in Section 2.2, the first step to obtain a pair of collision messages is to construct a collision trail with a high differential probability. With a good collision trail, the dominant complexity is then to find message pairs that follow it. Once such message pairs are obtained, generating colliding messages becomes straightforward.

A Valid Collision Attack. In differential-based collision attacks against sponge hash functions, a two-phase strategy is commonly used. The first phase, known as the *connecting phase*, involves generating message pairs that satisfy the conditions specified by the connecting trail. Such conditions are called the connecting conditions, and the complexity of this phase is denoted by $T_{\text{connecting}}$. The second phase, called the *colliding phase*, with its time complexity denoted by $T_{\text{colliding}}$, focuses on generating message pairs that follow the colliding trail. This approach reduces the total time complexity from the naive estimate of $T_{\text{connecting}} \times T_{\text{colliding}}$ to $T_{\text{connecting}} + T_{\text{colliding}}$.

In the 2-round Ascon-Hash256 collision trail, the differential of the active Sbox in the connecting trail (with a weight of $n_c = 54$) follows the pattern (10000,**0**). This implies that the input values of the connecting trail must satisfy Equation 6. More specifically, there are n_c connecting conditions that apply to the inner part of state S_{s-1} in the attack framework shown in Figure 2.

When the collision trail is introduced at block M_{s-1} , the n_c conditions on the inner part of S_{s-1} can only be fulfilled using the r degrees of freedom of block M_{s-2} . Given that $n_c < r$, probabilistically, if we try 2^{n_c} messages from the 2^r possible messages in block M_{s-2} , we can expect to find at least one message that follows the connecting trail, which is referred to as a *valid* M_{s-2} message.

Once we obtain a valid M_{s-2} message, the next step is to find message pairs that follow the colliding trail. As the colliding trail has a weight of n_d , by exhaustive search, we need to check approximately 2^{n_d} message pairs $((M_{s-2}, M_{s-1}), (M_{s-2}, M'_{s-1}))$ before finding a collision pair. With the degrees of freedom of block M_{s-1} , each valid M_{s-2} message allows for 2^r such pairs, all of which follow the connecting trail. Thus, we need to generate 2^{n_d-r} valid M_{s-2} messages to obtain a collision message.

To elaborate, each M_{s-2} block can provide 2^{r-n_c} valid M_{s-2} messages, and each valid message requires 2^{n_c} computations of the 2-round Ascon permutation. Consequently, generating 2^{n_d-r} valid M_{s-2} messages necessitates checking approximately $2^{n_c+n_d-2r}$ M_{s-2} blocks. Note that each M_{s-3} message represents one such block M_{s-2} with r degrees of freedom, so we need $2^{n_c+n_d-2r}$ M_{s-3} messages.

Therefore, the total complexity of generating 2^{n_d-r} valid messages (M_{s-3}, M_{s-2}) that follow the connecting trail is:

$$\begin{aligned} T_{\text{connecting}} &= 2^{n_c+n_d-2r} + 2^{n_c} \cdot 2^{n_d-r} \\ &= 2^{92} \approx 2^{28} + 2^{54} \times 2^{38} \end{aligned}$$

2-round Ascon computation.

With the valid (M_{s-3}, M_{s-2}) messages, the message pairs that follow the collision trail, denoted by $((M_{s-3}, M_{s-2}, M_{s-1}), (M_{s-3}, M_{s-2}, M'_{s-1}))$, can be obtained by exhaustively searching through the 2^{n_d} possible message pairs. This requires:

$$T_{\text{colliding}} = 2^{n_d} = 2^{102}$$

2-round Ascon computations.

After obtaining the message pairs $((M_{s-3}, M_{s-2}, M_{s-1}), (M_{s-3}, M_{s-2}, M'_{s-1}))$ that follow the collision trail, we can choose any pair of messages (M_s, M'_s) with a difference identical to the output difference of the collision trail. This allows us to directly obtain a pair of collision messages, i.e., $((M_{s-3}, M_{s-2}, M_{s-1}, M_s), (M_{s-3}, M_{s-2}, M'_{s-1}, M'_s))$. Thus, the total time complexity for the collision attack is:

$$T = T_{\text{connecting}} + T_{\text{colliding}} = 2^{102} \approx 2^{92} + 2^{102}$$

2-round Ascon permutation.

In this attack, $s = 3$ is determined because block M_{s-3} corresponds to block M_0 , indicating a 4-block collision attack where the collision trail is introduced at block M_2 .

Blocks M_0 and M_1 provide the necessary degrees of freedom to generate messages that satisfy the connecting conditions. By using the *connecting* and *colliding* method, we can mount a valid collision attack on the 2-round Ascon-Hash256 with complexity lower than the security bound. Practically, this method has been employed in the analysis of the 2-round Ascon-Hash256 collision in [GPT21].

Note that although exhaustive search is still used in both the connecting and colliding phases, the total time complexity is significantly reduced. In the following Section D.3, we will introduce optimization techniques that further accelerate the connecting phase, demonstrating that the *connecting* and *colliding* strategy addresses the CICO problem effectively.

D.3 Enhanced 2-Round Collision Attack

In this section, we build upon the attack framework presented in Section 2.2 and the detailed model of a valid collision attack outlined in Section D.2 to demonstrate how the connecting conditions can be satisfied more efficiently by exploiting specific properties of the Ascon permutation. Note that these optimization techniques are directly taken from [YLW⁺23]. We review this work to investigate potential improvements to the current collision attack.

Figure 14 is used to illustrate the 4-block collision attack, highlighting the value propagation throughout the process. For each permutation, x_k represents the input state to the k -th round Sbox operation, and y_k represents the input state to the k -th round linear L operation, where $k \in \{0, 1, 2\}$ for x_k and $k \in \{0, 1\}$ for y_k . The j -th row of the state x_k (or y_k), which is the 64-bit register word of the state, is denoted by x_k^j (or y_k^j), where $j \in \{0, 1, 2, 3, 4\}$. Within this row, the i -bit is denoted by $x_k^{j,i}$ (or $y_k^{j,i}$), where $i \in \{0, \dots, 63\}$. Since the four inner part rows of each state x_0 , which corresponds to the input state to S_i to each permutation with $i \in \{0, \dots, s\}$, are constants determined by the previous message block, they are also denoted as $c_0^1 \sim c_0^4$.

Conditions in the Connecting and Colliding Phases. As discussed in Section D.2, at block M_2 , the connecting trail imposes $n_c = 54$ conditions on the inner part of state S_2 . Additionally, for any active α_1 Sbox (indexed by z) with the differential pattern $(00*00, 10000)$, we can deduce from Equation 7 that there is a condition on the second bit of the input to the z -th α_1 Sbox, i.e., $x_1^{z,2} = C_1^{z,2}$, where $C_1^{z,2}$ is a constant determined by the differential.

Moreover, as shown in the value propagation of block M_1 (see Figure 14), when the inner part of state S_1 is given, the second row of the state x_1 , denoted by x_1^2 , is entirely determined by the four inner part rows of S_1 , specifically row x_0^1 , x_0^2 , x_0^3 , and x_0^4 . This relationship can be expressed as: $x_1^2 = G(x_0^1, x_0^2, x_0^3, x_0^4)$, where G represents the functional relationship between these values. The same logic applies to state S_2 , indicating that the $d = 28$ conditions (as there are 28 active Sboxes) $x_1^{z,2} = C_1^{z,2}$ imposed by the colliding trail actually apply to the inner part of S_2 , just like the conditions imposed by the connecting trail.

Thus, during the connecting phase, there are a total of $82 = n_c + d = 54 + 28$ connecting conditions on the inner part of the S_2 state. The remaining conditions of the colliding trail are actually $74 = n_d - d = 102 - 28$, rather than the initially anticipated $n_d = 102$ (i.e., the weight of the connecting trail).

When checking whether the $n_c + d$ conditions are satisfied for a given M_0 message and the M_1 block with r degree of freedom, to avoid the inefficient exhaustive search, optimization techniques based on the analysis of value propagation in M_1 are used. With Figure 14, let's first examine the propagation of the degrees of freedom of block M_1 in terms of the algebraic degree of those M_1 variables.

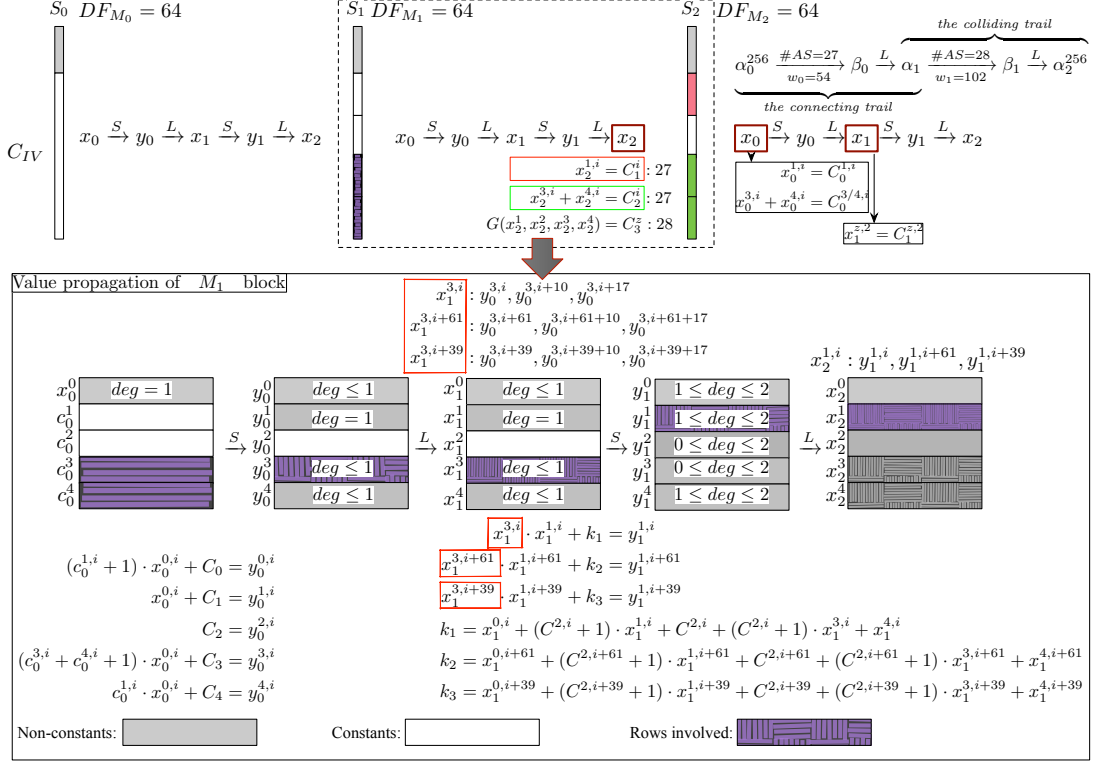


Figure 14: The 2-round collision attack against Ascon-Hash256.

The Algebraic Degree of the States in the Value Propagation of S_1 . Given the input state S_1 of a permutation, the degrees of freedom of block M_1 (denoted as *variables* and totaling $r = 64$) propagate through the following two rounds, as illustrated in Figure 14. The algebraic degree of each state x_i and y_i with respect to the M_1 variables is summarized in a row-wise manner, with $\deg(x_k^j)$ denotes the algebraic degree of row x_k^j .

- Algebraic degree of x_0 : $\deg(x_0^0) = 1$, and $\deg(x_0^j) = 0$ for $j \in \{1, 2, 3, 4\}$

For the input state x_0 , only the first row x_0^0 has an algebraic degree of 1, corresponding to the r M_1 variables. The remaining four rows (i.e., row x_0^j for $j \in \{1, 2, 3, 4\}$), which correspond to the inner part of the output of S_0 , are constants, and therefore also denoted by c_0^1, c_0^2, c_0^3 , and c_0^4 .

- Algebraic degree of y_0 : $\deg(y_0^1) = 1$, $\deg(y_0^2) = 0$, and $\deg(y_0^j) \leq 1$ for $j \in \{0, 3, 4\}$

The r M_1 variables in row x_0^0 propagate to y_0 through the Sbox operation defined in Equation 1. As depicted in Figure 14, where the direct relation between the rows of y_0 and the rows of x_0 is listed, the second row y_0^1 has a degree of 1, while the degrees of row y_0^0, y_0^3 , and y_0^4 can be either 1 or 0 depending on the specific values of the involved constants. The third row y_0^2 remains constant, as previously discussed.

- Algebraic degree of y_1 : $1 \leq \deg(y_1^i) \leq 2$, and $0 \leq \deg(y_1^j) \leq 2$, where $i \in \{0, 1, 4\}$ and $j \in \{2, 3\}$

The linear L operation does not change the degree of the x_1 state. After passing through the Sbox operation, the algebraic degree of each of the five rows of y_1 can vary from 0 to 2, depending on the exact values of the inner part of S_1 .

To better understand how the algebraic degree changes, Figure 14 illustrates the relationships between the rows of y_1 and x_1 , as well as between the rows of x_1 and y_0 . These relationships are directly deduced by the definition of the nonlinear Sbox and linear L operations of the Ascon permutation. The figure provides specific examples that show how these operations affect the propagation of algebraic degrees across different states.

Linearizing the Connecting Conditions. Now, let's revisit the connecting conditions imposed on the inner part of S_2 . The inner part of S_2 is equivalent to that of the x_2 state, which is the output of S_1 through two rounds. As discussed earlier and also explained in Figure 14, the combination of the connecting trail and the colliding trail imposes a total of $n_c + d$ conditions on the rows $x_2^1 \sim x_2^4$ of the output state x_2 .

Specifically, for the $n_c = 54$ conditions, there are (1) $\frac{c}{2} = 27$ *simple conditions* on bits $x_2^{1,i}$, which are fixed to constants, i.e., $x_2^{1,i} = C_1^i$; and (2) $\frac{c}{2} = 27$ *moderately complex conditions* that involve sums such as $x_2^{3,i} + x_2^{4,i} = C_2^i$. In both cases, i represents the index of the active Sboxes in the connecting trail. For the $d = 28$ colliding conditions, which involve linear combinations of bits across the four rows $x_2^1 \sim x_2^4$, i.e., $G(x_2^1, x_2^2, x_2^3, x_2^4) = C_3^z$, they are referred to as *complex conditions*, as they are too complex to be handled.

To check how to meet these connecting conditions, let's begin with the simple conditions $x_2^{1,i} = C_1^i$. Given that the algebraic degree of row x_2^1 satisfies $1 \leq \deg(x_2^1) \leq 2$, it's possible to achieve $\deg(x_2^{1,i}) = 1$ by imposing specific constraints.

As illustrated in Figure 14, the key terms contributing to $x_2^{1,i}$ include $x_1^{3,i} \cdot x_1^{1,i}$, $x_1^{3,i+61} \cdot x_1^{1,i+61}$, and $x_1^{3,i+39} \cdot x_1^{1,i+39}$. Since $\deg(x_1^1) = 1$, to get $\deg(x_2^{1,i}) = 1$, we need to set the related terms $x_1^{3,i+C_i}$ as constants. This, in turn, implies that the associated terms y_0^{3,C_j} must also be constants, where C_i and C_j represent the related rotational constants of the L operation.

There are two ways to ensure that $y_0^{3,i}$ is constant. On the one hand, if $c_0^{3,i} + c_0^{4,i} = 1$, then $y_0^{3,i} = (c_0^{3,i} + c_0^{4,i} + 1) \cdot x_0^{0,i} + C_3^i$ will automatically be constant. This requires imposing additional conditions on the inner part of S_1 . Consequently, we need to prepare the valid M_0 messages in such a way that these conditions (i.e., $c_0^{3,i} + c_0^{4,i} = 1$) are satisfied. On the other hand, if we don't control the value of $c_0^{3,i} + c_0^{4,i}$, i.e., say $y_0^{3,i} = x_0^{0,i} + C_3^i$, we should consume the degrees of freedom of x_0^0 (or M_1) to make $y_0^{3,i}$ constant. In this case, we set $x_0^{0,i} = 0$ or $x_0^{0,i} = 1$ to achieve this¹⁰.

When $y_0^{3,i}$ is set to a constant using either of the two methods, $x_2^{1,i}$ becomes linear, transforming the condition $x_2^{1,i} = C_1^i$ into a linear equation involving x_0^0 . This linearization allows us to resist exhaustive search. Instead, we can efficiently find messages that satisfy these conditions by formulating and solving the resulting linear equations.

D.4 Initial SAT Implementation for Collision Trail Search

The best-known 2-round collision trail with a weight of 156, was given by Gerault *et al.* in [GPT21] using a CP-based automatic search tool. While collision trails for 4-round Semi-Free-Start (SFS) collision attacks [DEMS19, YLW⁺23] have been reported, no trails promising valid collision attacks have been presented. In this section, we construct SAT

¹⁰In this scenario, since $y_0^{3,i}$ and $y_0^{1,i}$ are both linear functions of x_0^0 , setting either to be constant would have the same effect. Here, $y_0^{3,i}$ is used as an example.

models to search for the collision trails. As discussed in Section 2.3, the encodings includes the techniques that translate the truth table to CNFs, and the cardinality encoding.

The SAT Collision Trail Search Model. An r -round collision trail is shown below,

$$\alpha_0^{256} \xrightarrow{S} \boxed{\beta_0} \xrightarrow{L} \alpha_1 \xrightarrow{S} \cdots \xrightarrow{S} \beta_{r-2} \xrightarrow{L} \boxed{\alpha_{r-1}} \xrightarrow{S} \beta_{r-1} \xrightarrow{L} \alpha_r^{256}.$$

the SAT model

As the input and output difference constraints are directly transformed into constraints on β_0 and α_{r-1} , the SAT model covers $(r - 1.5)$ rounds, beginning with β_0 and ending with α_{r-1} .

With each variable corresponding to one bit of the state, a general SAT-based collision trail search model includes a CNF description of difference propagation, state constraints, and an objective function. Specifically, the variables in the SAT model can be categorized into three types. *State variables* represent the state difference, with each state being modeled by 320 variables. *Parameter variables* are used to indicate cryptanalytic properties such as the activeness of an Sbox or the weights of Sboxes. Finally, *auxiliary variables* are introduced as a result of cardinality encoding, which helps manage constraints related to objective functions, e.g., the number of active Sboxes.

The constraint model. The differential propagation (including the round operations p_S and p_L), the constraints on differences, and parameter definitions, are encoded using the truth-table minimization. In specific, the 3-bit XOR operation, the DDT of Sbox, the constraints on states, (e.g., fixing β_0 and α_{r-1} to specific sets), and the definition of Sbox activeness and weight are all encoded with this method. The Sympy library is employed to generate these CNF clauses.

The objective function model. The objective function is defined as the number of active Sboxes, which is translated into CNF clauses using kmtotalizer cardinality encoding.

2-Round Collision Trail Search. The 2-round collision trail search reduces to finding the half-round trail $\beta_0 \xrightarrow{L} \alpha_1$, as illustrated with

$$\alpha_0^{256} \xrightarrow{S} \boxed{\beta_0 \xrightarrow{L} \alpha_1} \xrightarrow{S} \beta_1 \xrightarrow{L} \alpha_2^{256}.$$

The Sbox differences of both α_0 and α_2 are fixed to 10000 (see Section D.1). Moreover, the Sbox differences of β_0 and α_1 are fixed to specific sets, i.e., $\beta_0 \in \{0, 9, 11, 24, 26\}$ and $\alpha_1 \in \{0, 3, 9, 18, 25\}$.

The SAT model describes the p_L operation between α_1 and β_0 , the constraints on β_0 and α_1 , and the objective function, which sets $AS(\beta_0) \leq X$ and $AS(\alpha_1) \leq Y$, where X and Y are determined from Table 8.

Using this SAT model, the known 2-round collision trail with weight 156 from [GPT21] can be reproduced within seconds. However, despite over 92,160 core hours (or 3840 core days) of computation, the model fails to find better solutions or produce an UNSAT result, suggesting that the current SAT model becomes stuck and is unlikely to solve this problem efficiently.

3/4/5-Round Collision Trail Search. For collision trails involving more than two rounds, the SAT model is constructed in a similar manner. It describes the basic propagation and constraints, and incorporates an objective function based on the number of active Sboxes.

Unfortunately, the efficiency of this approach deteriorates significantly for longer trails. Aside from trails suitable for SFS collision attacks, which are only marginally better than previously reported results, the SAT model again gets stuck, failing to produce better solutions.

D.5 Semi-Free-Start Collision Trail and Message Pair

This section provides the detailed differential trail and concrete message pair for the 3-/4-/5-round Semi-Free-Start (SFS) collision attack on **Ascon-Hash256** discovered using our SAT-based space partitioning technique.

D.5.1 3-Round SFS Collision Trail and Concrete Message Pair

Table 21 presents the complete differential trail for the 3-round SFS collision with active Sbox configuration (7, 25, 35) and total weight 216. Table 22 provides the actual message pair that follows the differential trail and produces a collision after 3 rounds of the Ascon permutation.

Table 21: 3-Round SFS collision trail for **Ascon-Hash256**

Round(i)	Probability	Input difference(α_{i-1})	Output difference(β_{i-1})
1	2^{-14}	0x4000a00502001000	0x0000000000000000
		0x0000000000000000	0x4000a00502001000
		0x0000000000000000	0x0000000000000000
		0x0000000000000000	0x0000200002001000
		0x0000000000000000	0x4000a00502001000
2	2^{-90}	0x0000000000000000	0x0000000100a09024
		0x4a01a00d12809142	0x4a00814108a48132
		0x0000000000000000	0x0000800c10a41146
		0x0800200812009104	0x4001204802849032
		0x4201a14d08241070	0x0801814108040016
3	2^{-112}	0x180d824100a0b020	0x9c9dc24ba9b4b9f6
		0x9815c24b291489a2	0x0000000000000000
		0x1800c20a28b489a0	0x0000000000000000
		0x0488000080a03054	0x0000000000000000
		0x84958043811008d6	0x0000000000000000

Table 22: 3-Round SFS collision message pair for **Ascon-Hash256**

Message M_1	Message M'_1	Output H_1	Output H'_1
0xbb794a023a6af9bb	0xfb79ea07386ae9bb	0x88d0ba1c34f46e8b	0x183834adecd586f1
0x5d18bbcd0b5412ad	0x5d18bbcd0b5412ad	0xbe9644f347480fa9	0xbe9644f347480fa9
0x6dc0f2c08ec9e4e3	0x6dc0f2c08ec9e4e3	0x84e32322e8105548	0x84e32322e8105548
0xbf6020971e584875	0xbf6020971e584875	0x2b71112875c19a99	0x2b71112875c19a99
0xe208f3709385e181	0xe208f3709385e181	0xb21b42149a8bdda7	0xb21b42149a8bdda7

D.5.2 4-Round SFS Collision Trail and Concrete Message Pair

Table 23 presents the complete differential trail for the 4-round SFS collision with the number of active Sbox configuration (3, 11, 31, 33) and total weight 250. Table 24 provides

the actual message pair that follows the differential trail and produces a collision after 4 rounds of the Ascon permutation.

Table 23: 4-Round SFS collision trail for Ascon-Hash256

Round(i)	Probability	Input difference(α_{i-1})	Output difference(β_{i-1})
1	2^{-6}	0x0100001000004000	0x0000001000004000
		0x0000000000000000	0x0100001000004000
		0x0000000000000000	0x0000000000000000
		0x0000000000000000	0x0000000000004000
		0x0000000000000000	0x0100000000000000
2	2^{-35}	0x0804001000024100	0x000200100000c010
		0x2900001000004000	0x090600100002c100
		0x0000000000000000	0x2900001000000000
		0x2000000000004010	0x080400000000110
		0x0102000000008000	0x0904000000020110
3	2^{-103}	0x180c01104022c110	0x0482090080000200
		0x613605120006c500	0x09280d128826c702
		0x3d24001840000000	0x798801084000c410
		0x4c8e050200000110	0x50960c1800040102
		0x2916090088068712	0x749c000208208502
4	2^{-106}	0x04c2299089008208	0x75fb29d8cd0aaf08
		0x65782808cd02af08	0x0000000000000000
		0x04aa21884100a508	0x0000000000000000
		0x108301d000080000	0x0000000000000000
		0x717128408d0a8a08	0x0000000000000000

Table 24: 4-Round SFS collision message pair for Ascon-Hash256

Message M_1	Message M'_1	Output H_1	Output H'_1
0xc0a9635592074a11	0xc1a9634592070a11	0xfd532355dcabb298	0x0de3f4b52b2899bd
0x031f1a6fcc2e13c3	0x031f1a6fcc2e13c3	0x5c599ce3bfd4ebb9	0x5c599ce3bfd4ebb9
0x79d00b4065f34dfb	0x79d00b4065f34dfb	0x8680d74a16f1a68f	0x8680d74a16f1a68f
0xb280b586aa2e1ed9	0xb280b586aa2e1ed9	0x08ddf2dd249eef82	0x08ddf2dd249eef82
0x8f007bbc7bd53580	0x8f007bbc7bd53580	0x6a3c77d29cb841d0	0x6a3c77d29cb841d0

D.5.3 5-Round SFS Collision Trails

Table 25 presents the first 5-round collision trail with the number of active Sbox configuration (1, 5, 18, 33, 35) and total weight 298. Table 26 presents the second 5-round collision trail with the number of active Sbox configuration (3, 11, 17, 31, 34) and total weight 310.

Table 25: 5-Round SFS collision trail 1 for Ascon-Hash256 (weight 298)

Round(i)	Probability	Input difference(α_{i-1})	Output difference(β_{i-1})
1	2^{-2}	0x0000001000000000	0x0000000000000000
		0x0000000000000000	0x0000001000000000
		0x0000000000000000	0x0000000000000000
		0x0000000000000000	0x0000000000000000
		0x0000000000000000	0x0000001000000000
2	2^{-16}	0x0000000000000000	0x0000000000000000
		0x2000009000000000	0x2800000020000000
		0x0000000000000000	0x2000008000000000
		0x0000000000000000	0x2000008000000000
		0x0800001020000000	0x2800009020000000
3	2^{-56}	0x0000000000000000	0x2088100000040001
		0x6840000120500001	0x3000101020440001
		0x308000c200000000	0x188800c200000001
		0x2008108020400000	0x188000c220400001
		0x6040009100540000	0x6048109100100000
4	2^{-114}	0xa0c834030a850001	0xa0c83433120c0021
		0x9040189120040028	0x080898b128090408
		0x90ae20a008000001	0x0828b8a338ad0021
		0x18c6ac8210a90021	0x0022081000000000
		0x280888b022200408	0x38ce803132a90401
5	2^{-110}	0x000c023018092151	0x2a3e4fb378593579
		0x2a1c4f3078513579	0x0000000000000000
		0x081c461028193431	0x0000000000000000
		0x0022008300080000	0x0000000000000000
		0x222649b350503149	0x0000000000000000

Table 26: 5-Round SFS collision trail 2 for Ascon-Hash256 (weight 310)

Round(i)	Probability	Input difference(α_{i-1})	Output difference(β_{i-1})
1	2^{-6}	0x04800000000010000	0x0000000000000000
		0x0000000000000000	0x04800000000010000
		0x0000000000000000	0x0000000000000000
		0x0000000000000000	0x0400000000000000
		0x0000000000000000	0x04800000000010000
2	2^{-35}	0x0000000000000000	0x0000000000000000
		0x2080020000000000	0x0409028000034200
		0x0000000000000000	0x2080000000000000
		0x0401020000000000	0x2008000000000000
		0x0489008000034200	0x2409028000034200
3	2^{-59}	0x0000000000000000	0x4400020000014000
		0x2441100400114005	0x6000132000004004
		0x3042000000000000	0x3002010000000001
		0x2000120400000000	0x1442110400000000
		0x6441112400114005	0x2041032000104000
4	2^{-100}	0x6c140a8440416000	0x9492000309806000
		0x20008aa008c24001	0xfc14838749412000
		0xacc3098400000001	0x1c840ba308c20000
		0x14470ba149820000	0x4085832248020001
		0xb001890640004001	0x4047022509030001
5	2^{-110}	0x0094129800a00100	0x52f71ffd43b12d80
		0x12221dfd02b00900	0x0000000000000000
		0x12b41e5c00800800	0x0000000000000000
		0x40d5020041012480	0x0000000000000000
		0x50430da143b12580	0x0000000000000000

E Computation of Interdependent Values in MitM Collision

In Algorithm 1, when processing red and blue state values (Lines 7-15), we encounter interdependent bits where red value calculations depend on $\blacksquare/\blacksquare$ constants, and vice versa. This section describes a pre-computation approach for efficiently handling these dependencies.

Computation of Values in y_1 . Consider the example from Figure 10, where input bits of the 30th Sbox in state x_1 are computed from related y_0 bits through the p_L operation:

$$\begin{aligned} x_1^{30,0} &= y_0^{30,0} + y_0^{26,0} + y_0^{17,0} = c_0^r + c_0^b \\ x_1^{30,1} &= y_0^{30,1} + y_0^{5,1} + y_0^{27,1} = c_1^B \\ x_1^{30,2} &= c_2 \\ x_1^{30,3} &= y_0^{30,3} + y_0^{15,3} + y_0^{8,3} = c_3^b \\ x_1^{30,4} &= y_0^{30,4} + y_0^{7,4} + y_0^{5,4} = c_4^r + c_4^b \end{aligned}$$

The colors correspond to the dependencies of each bit. And $x_k^{i,j}$ and $y_k^{i,j}$ represent the j -th bit of the i -th Sbox in states x_k and y_k , respectively.

We denote c_m^r as the red-dependent part of a bit and c_m^b as the blue-dependent part. The constant c_m represents values determined by the $\blacksquare$ value in y_0 . Furthermore, c_m^B denotes the constant corresponding to $\blacksquare$ or $\blacksquare$ bits, while c_m^R denotes the constant corresponding to $\blacksquare$ or $\blacksquare$ bits. Here $m \in \{0, 1, 2, 3, 4\}$ represents the bit index of the Sbox.

When these values propagate through the Sbox operation, the output bits involved in further matching (i.e., the $y_1^{30,0}$ and $y_1^{30,1}$ bits) contain both red-dependent and blue-dependent components.

$$\begin{aligned} y_1^{30,0} &= x_1^{30,1} \cdot (x_1^{30,0} + x_1^{30,4}) + c_2 \cdot (x_1^{30,1} + 1) + x_1^{30,0} + x_1^{30,1} + x_1^{30,3} \\ &= \begin{cases} c_1^B \cdot (c_0^b + c_4^b) + c_2 \cdot (c_1^B + 1) + c_0^b + c_1^B + c_3^b & \leftarrow \text{blue-dependent part} \\ c_1^B \cdot (c_0^r + c_4^r) + c_0^r & \leftarrow \text{red-dependent part} \end{cases} \\ y_1^{30,1} &= x_1^{30,1} \cdot x_1^{30,3} + c_2 \cdot (x_1^{30,1} + x_1^{30,3} + 1) + x_1^{30,0} + x_1^{30,1} + x_1^{30,3} + x_1^{30,4} \\ &= \begin{cases} c_1^B \cdot c_3^b + c_2 \cdot (c_1^B + c_3^b + 1) + c_0^b + c_1^B + c_3^b + c_4^b & \leftarrow \text{blue-dependent part} \\ c_0^r + c_4^r & \leftarrow \text{red-dependent part} \end{cases} \end{aligned}$$

For each $\blacksquare$ value v_B , the blue-dependent parts are computed directly. Conversely, for each $\blacksquare$ value v_R , the computation of red-dependent parts (such as $y_1^{30,0}$) may involve blue constants c_1^B .

In Figure 10, among the 70 involved $\blacksquare$ and $\blacksquare$ bits in state y_1 , exactly 12 bits depend on $\blacksquare/\blacksquare$ constants. Importantly, each of these 12 bits depends on only a single $\blacksquare/\blacksquare$ bit. To efficiently handle these dependencies, we pre-compute values for both 0 and 1 of each dependent constant.

For example, when calculating the red-dependent part of $y_1^{30,0}$ which depends on blue constant c_1^B , we compute results for both $c_1^B = 0$ and $c_1^B = 1$. As we can see from the above equation, computing the result for one c_1^B value automatically yields the result for the other without additional computation. Thus, the pre-computation strategy introduces no extra computational overhead.

Now, to apply this strategy when computing the matching values m_R and m_B in state x_2 , we first need to understand the matching conditions used in the MitM collision attack.

Computation of Matching Values. In Figure 10, matching conditions on red and blue values are applied to the corresponding Sbox input bits in state x_2 . Equation 8 presents

the exact matching conditions for these bits. The equation contains 32 expressions, predominantly combining red and blue terms, with 8 expressions containing exclusively red or blue terms.

$$\begin{aligned}
& x_2^{22,0,r} + x_2^{22,0,b} = 1 & x_2^{11,0,r} + x_2^{11,0,b} = 1 & x_2^{1,0,r} + x_2^{1,0,b} = 1 & x_2^{0,0,r} + x_2^{0,0,b} = 1 \\
& x_2^{22,1,r} + x_2^{22,2,r} + x_2^{22,1,b} + x_2^{22,2,b} = 1 & x_2^{11,1,r} + x_2^{11,2,r} + x_2^{11,1,b} + x_2^{11,2,b} = 1 & x_2^{1,1,r} + x_2^{1,2,r} + x_2^{1,1,b} + x_2^{1,2,b} = 1 & x_2^{0,1,r} + x_2^{0,2,r} + x_2^{0,1,b} + x_2^{0,2,b} = 1 \\
& x_2^{22,3,r} = 0 & x_2^{11,3,b} = 0 & x_2^{1,3,b} = 0 & x_2^{0,3,r} = 0 \\
& x_2^{22,4,r} + x_2^{22,4,b} = 0 & x_2^{11,4,r} + x_2^{11,4,b} = 0 & x_2^{1,4,r} + x_2^{1,4,b} = 0 & x_2^{0,4,r} + x_2^{0,4,b} = 0
\end{aligned} \tag{8}$$

In detail, among these 32 matching positions, red-dependent values appear in 28 conditions, with 4 conditions involving only red values (forming a 4-bit filter). Similarly, blue-dependent values appear in 28 conditions, with 4 conditions involving only blue values (forming another 4-bit filter). These red and blue terms are computed directly from the involved bits in y_1 .

As discussed earlier, some of these involved y_1 bits depend on $\blacksquare/\blacksquare$ or $\blacksquare/\blacksquare$ constants. Specifically, 12 red terms in Equation 8 depend on $\blacksquare/\blacksquare$ bits, and 12 blue terms depend on $\blacksquare/\blacksquare$ bits, with each term depending on precisely one constant bit.

When computing the matching values in state x_2 , we apply the same pre-computation strategy. For clarity, we refer to these values as the red matching value $m_{\mathcal{R}}$ and the blue matching value $m_{\mathcal{B}}$.

For each red value $v_{\mathcal{R}}$, we store the entry $(v_{\mathcal{R}}, m_{\mathcal{R}})$ in appropriate tables. The matching value $m_{\mathcal{R}}$ consists of two distinct components: for terms dependent on $\blacksquare/\blacksquare$ bits, we store two possible values (one for each possible constant value); for independent terms, we store only one value. We follow the same procedure for blue values and their matching values.

This pre-computation strategy offers two key advantages. First, it eliminates redundant computation during the matching phase by enabling retrieval of pre-computed values. Second, it incurs no additional computational overhead despite storing multiple values for dependent terms, as the calculation for one constant value automatically yields the result for the alternative value. During matching, direct lookup based on the specific constants involved significantly enhances the overall efficiency of the process.